```
// ============================================================================
// 文件：entry/src/main/ets/common/Cms.ets     （共 703 行）
// ============================================================================
import hilog from '@ohos.hilog';
import { Http } from './Http';
import { Site, Category, Vod, VodPage, Line, Episode, JResp, JVod, JClass,
  FilterGroup, FilterValue } from './Model';
/**
 * 内容接口客户端：面向标准 CMS 开放数据接口。
 *
 * 支持三类数据源协议：
 *   type 0 —— XML 数据接口
 *   type 1 —— JSON 数据接口
 *   type 4 —— 返回体多包一层 data 的 JSON 接口（同一套参数）
 * 脚本型实现依赖 JVM 与字节码加载器，本平台不支持，由上层标记为不可用。
 */
/** 一条接口响应缓存。key 即完整 URL（含接口地址与查询串），天然按来源隔离 */
class CmHit {
  key: string = '';
  body: string = '';
  at: number = 0;
}
export class Cms {
  /**
   * 接口响应缓存。
   *
   * 一次列表请求往往 1~3 秒，而用户在分类标签上来回点的概率很高
   * （看看「电影」有没有新片、再回「电视剧」继续）。之前每次切换都重新请求，
   * 回来一次就白等一次。这里加一层按 URL 的短 TTL 缓存：
   *   分类表 5 分钟（基本不变）
   *   列表   90 秒（够「来回切」，又不会把刚更新的片单压太久）
   *   详情   10 分钟（详情几乎不变，但剧集会更新，所以不给太长）
   * 搜索不缓存：关键词千变万化，缓存命中率低，还容易把「刚试过的搜索」结果留旧。
   */
  private static hits: CmHit[] = [];
  /**
   * 最多缓存条数（LRU 淘汰）。用户长时间浏览多个站点时不会无限增长：
   * 100 条约 ~6–10 MB，超出后淘汰最久未访问的，不是最早写入的。
   */
  private static readonly HIT_MAX: number = 100;
  /** 单条响应最大字符数；单个超大响应不独占缓存 */
  private static readonly HIT_SINGLE_MAX: number = 500000;
  /** 缓存总字符上限；超出从最久未访问的开始淘汰 */
  private static readonly HIT_CHARS: number = 5000000;
  private static readonly TTL_CAT: number = 300000;
  private static readonly TTL_LIST: number = 90000;
  private static readonly TTL_DETAIL: number = 600000;
  /**
   * 读缓存（LRU 命中）：命中时把项目移到数组尾部（最近使用），
   * 淘汰时从头部取（最久未使用），实现真 LRU。
```

```
   */
  private static hit(url: string, ttl: number): string {
    const now: number = Date.now();
    for (let i = 0; i < Cms.hits.length; i++) {
      if (Cms.hits[i].key === url) {
        if (now - Cms.hits[i].at <= ttl) {
          // LRU：命中时把这条移到尾部，标记为最近使用
          const item: CmHit = Cms.hits[i];
          Cms.hits.splice(i, 1);
          Cms.hits.push(item);
          return item.body;
        }
        Cms.hits.splice(i, 1);   // 过期就地清掉
        return '';
      }
    }
    return '';
  }
  /**
   * 写缓存（LRU 淘汰）：超容时从头部（最久未用）依次丢，而不是固定丢最旧写入的。
   * 超大单条响应（>500KB）不缓存，避免一条大页占满整个缓存池。
   */
  private static keep(url: string, body: string): void {
    if (body.length === 0) {
      return;                  // 空响应不缓存，否则一次失败会「粘」住 90 秒
    }
    if (body.length > Cms.HIT_SINGLE_MAX) {
      hilog.info(0x1000, 'BOXN', 'cache skip (too large %{public}d chars)', body.l
ength);
      return;
    }
    const e = new CmHit();
    e.key = url;
    e.body = body;
    e.at = Date.now();
    // 同 URL 覆盖（先删再 push 到尾部）
    for (let i = 0; i < Cms.hits.length; i++) {
      if (Cms.hits[i].key === url) {
        Cms.hits.splice(i, 1);
        break;
      }
    }
    Cms.hits.push(e);
    // LRU：超条数时淘汰头部（最久未访问）
    while (Cms.hits.length > Cms.HIT_MAX) {
      Cms.hits.shift();
    }
    // 总字符超限时，也从头部淘汰
    let total: number = 0;
    for (let i = 0; i < Cms.hits.length; i++) {
```

```
      total += Cms.hits[i].body.length;
    }
    while (total > Cms.HIT_CHARS && Cms.hits.length > 1) {
      total -= Cms.hits[0].body.length;
      Cms.hits.shift();
    }
  }
  /**
   * 缓存透明的一层：命中直接返回，未命中请求后落缓存。
   * 注意不能叫 text —— 类里已有一个 `Cms.text(s)` 负责剥 HTML 标签。
   */
  private static async fetch(url: string, ttl: number, ms: number): Promise<string
> {
    const has: string = Cms.hit(url, ttl);
    if (has.length > 0) {
      hilog.info(0x1000, 'BOXN', 'cache HIT  %{public}s', Cms.short(url));
      return has;
    }
    const t0: number = Date.now();
    const txt: string = await Http.text(url, undefined, ms);
    hilog.info(0x1000, 'BOXN', 'cache MISS %{public}s  %{public}d ms',
      Cms.short(url), Date.now() - t0);
    Cms.keep(url, txt);
    return txt;
  }
  /** 日志里只留参数部分，URL 太长会把 hilog 刷屏 */
  private static short(url: string): string {
    const i: number = url.indexOf('?');
    const q: string = i >= 0 ? url.substring(i + 1) : url;
    return q.length > 48 ? q.substring(0, 48) : q;
  }
  /** 用户更换配置后主动清空，避免旧响应还留着 */
  static clearCache(): void {
    Cms.hits = [];
  }
  /** 接口缓存里有几条（设置页「清理缓存」展示用） */
  static cacheCount(): number {
    return Cms.hits.length;
  }
  /** 接口缓存占用的字符数，1 个中文字符约 3 字节，显示时按 3 倍折算 */
  static cacheChars(): number {
    let n: number = 0;
    for (let i = 0; i < Cms.hits.length; i++) {
      n += Cms.hits[i].body.length;
    }
    return n;
  }
  private site: Site;
  /** 筛选条件尾巴，形如 &area=大陆&year=2025。由点播页切换筛选时塞进来 */
  private extra: string = '';
```

```
  constructor(site: Site) {
    this.site = site;
  }
  setExtra(q: string): void {
    this.extra = q;
  }
  // ---------- 公共入口 ----------
  async categories(): Promise<Category[]> {
    const txt: string = await Cms.fetch(this.u('ac=list'), Cms.TTL_CAT, 15000);
    if (this.site.type === 0 || Cms.looksXml(txt)) {
      const cats: Category[] = Cms.catFromXml(txt);
      if (cats.length > 0) {
        const all = new Category();
        all.id = '';
        all.name = '全部';
        cats.unshift(all);
        return cats;
      }
      // 该站的 ?ac=list 直接返回影片列表，只能从条目里反推分类；
      // 这时补一个「全部」(tid 传空串接口会返回全集)，否则会被锁死在单个分类里
      const derived: Category[] = Cms.catsFromVideos(txt);
      if (derived.length > 0) {
        const all = new Category();
        all.id = '';
        all.name = '全部';
        derived.unshift(all);
      }
      return derived;
    }
    const root: JResp = Cms.root(Cms.parse(txt));
    const out: Category[] = [];
    if (root.catList !== undefined) {
      for (let i = 0; i < root.catList.length; i++) {
        const c = new Category();
        c.id = String(root.catList[i].type_id === undefined ? '' : root.catList[i]
.type_id);
        c.name = Cms.s(root.catList[i].type_name);
        // type_pid 为 0 / 缺省都算顶级分类
        const pidAny: number | undefined = root.catList[i].type_pid;
        const pidRaw: number = pidAny === undefined ? 0 : pidAny;
        c.pid = pidRaw > 0 ? String(pidRaw) : '';
        if (c.id.length > 0 && c.name.length > 0) {
          out.push(c);
        }
      }
    }
    if (out.length === 0 && root.list !== undefined) {
      const seen: string[] = [];
      for (let i = 0; i < root.list.length; i++) {
        const tid: string = String(root.list[i].tid === undefined ? '' : root.list
```

```
[i].tid);
        const tn: string = Cms.s(root.list[i].type_name);
        if (tid.length > 0 && tn.length > 0 && seen.indexOf(tid) < 0) {
          seen.push(tid);
          const c = new Category();
          c.id = tid;
          c.name = tn;
          out.push(c);
        }
      }
    }
    if (out.length > 0) {
      // t 传空串多数 CMS 会返回全集，作为默认入口体验更好：
      // 部分站点的分类 id 里只有零星几条，不开「全部」会让人以为没内容
      const all = new Category();
      all.id = '';
      all.name = '全部';
      out.unshift(all);
    }
    return out;
  }
  async list(tid: string, page: number): Promise<VodPage> {
    const q: string = 'ac=videolist&t=' + encodeURIComponent(tid) + '&pg=' + page
+ this.extra;
    const txt: string = await Cms.fetch(this.u(q), Cms.TTL_LIST, 18000);
    if (this.site.type === 0 || Cms.looksXml(txt)) {
      return Cms.listFromXml(txt, page, this.site);
    }
    const root: JResp = Cms.root(Cms.parse(txt));
    const pg = new VodPage();
    pg.page = root.page === undefined ? page : root.page;
    pg.pageCount = root.pagecount === undefined ? 1 : root.pagecount;
    pg.total = root.total === undefined ? 0 : root.total;
    if (root.list !== undefined) {
      for (let i = 0; i < root.list.length; i++) {
        pg.items.push(this.toVod(root.list[i]));
      }
    }
    return pg;
  }
  /**
   * 把一个父分类和它的子分类合在一起取第一页。
   *
   * 为什么需要：父分类（如 `电视剧`）通常**没有直属影片**，
   * 直接请求 `t=1` 返回空列表，界面就会显示「这个分类下没有内容」——
   * 但下面明明排着 欧美剧/韩剧/日剧。用户会以为软件坏了。
   * 这里把父 + 子同时请求、按 vod id 去重合并，父分类点进去也有内容。
   *
   * 只取第 1 页且不做翻页：聚合结果本来就是「各类最新的混排」，
   * 继续翻页的语义说不清（翻谁的页？），不如把翻页留给具体的子分类。
```

```
   */
  async listMerged(ids: string[], page: number): Promise<VodPage> {
    const out = new VodPage();
    out.page = 1;
    out.pageCount = 1;
    const seen: string[] = [];
    const tasks: Promise<VodPage>[] = [];
    for (let i = 0; i < ids.length; i++) {
      tasks.push(this.list(ids[i], page));
    }
    const got: VodPage[] = await Promise.all(tasks);
    for (let i = 0; i < got.length; i++) {
      const items: Vod[] = got[i].items;
      for (let k = 0; k < items.length; k++) {
        if (seen.indexOf(items[k].id) < 0) {
          seen.push(items[k].id);
          out.items.push(items[k]);
        }
      }
    }
    out.total = out.items.length;
    return out;
  }
  async detail(id: string): Promise<Vod> {
    const q: string = 'ac=detail&ids=' + encodeURIComponent(id);
    const txt: string = await Cms.fetch(this.u(q), Cms.TTL_DETAIL, 18000);
    if (this.site.type === 0 || Cms.looksXml(txt)) {
      return Cms.detailFromXml(txt, this.site, id);
    }
    const root: JResp = Cms.root(Cms.parse(txt));
    if (root.list !== undefined && root.list.length > 0) {
      return this.toVod(root.list[0]);
    }
    throw new Error('接口没有返回详情数据');
  }
  /** 搜索；不同站对 ac / wd 的接受度不同，做两级兜底 */
  async search(kw: string): Promise<Vod[]> {
    const e: string = encodeURIComponent(kw);
    let items: Vod[] = [];
    try {
      const txt: string = await Http.text(this.u('ac=videolist&wd=' + e), undefine
d, 12000);
      items = this.pickList(txt);
    } catch (err) {
      items = [];
    }
    if (items.length === 0) {
      const txt2: string = await Http.text(this.u('wd=' + e), undefined, 12000);
      items = this.pickList(txt2);
    }
```

```
    return items;
  }
  private pickList(txt: string): Vod[] {
    if (this.site.type === 0 || Cms.looksXml(txt)) {
      return Cms.listFromXml(txt, 1, this.site).items;
    }
    const root: JResp = Cms.root(Cms.parse(txt));
    const out: Vod[] = [];
    if (root.list !== undefined) {
      for (let i = 0; i < root.list.length; i++) {
        out.push(this.toVod(root.list[i]));
      }
    }
    return out;
  }
  // ---------- 内部 ----------
  private u(params: string): string {
    const a: string = this.site.api;
    if (a.endsWith('?')) {
      return a + params;
    }
    if (a.indexOf('?') >= 0) {
      return a + '&' + params;
    }
    return a + '?' + params;
  }
  private toVod(j: JVod): Vod {
    const v = new Vod();
    v.id = String(j.vod_id === undefined ? '' : j.vod_id);
    v.name = Cms.s(j.vod_name);
    v.pic = Cms.s(j.vod_pic);
    v.remark = Cms.s(j.vod_remarks);
    v.year = Cms.s(j.vod_year);
    v.area = Cms.s(j.vod_area);
    v.actor = Cms.s(j.vod_actor);
    v.director = Cms.s(j.vod_director);
    v.content = Cms.plain(Cms.s(j.vod_content).length > 0 ? Cms.s(j.vod_content) :
 Cms.s(j.vod_blurb));
    v.typeName = Cms.s(j.type_name);
    // 分类 id 两个名字都可能出现：JSON 接口多为 type_id，XML 转出来的走 tid。
    // 站点返回数字或字符串都有，统一成字符串
    const tidAny: number | undefined = j.type_id !== undefined ? j.type_id : j.tid
;
    v.typeId = tidAny === undefined ? '' : String(tidAny);
    v.from = Cms.s(j.vod_play_from);
    v.playUrl = Cms.s(j.vod_play_url);
    v.siteKey = this.site.key;
    v.siteName = this.site.name;
    return v;
  }
```

```
// ---------- 静态工具 ----------
static s(v: string | undefined | null): string {
  if (v === undefined || v === null) {
    return '';
  }
  return String(v).trim();
}
static looksXml(txt: string): boolean {
  const t: string = txt.trimStart();
  return t.startsWith('<');
}
/** 配置 JSON 容错：去 BOM、去 // 注释行、把保留字 class 改名 */
static cleanJson(txt: string): string {
  let t: string = txt;
  if (t.charCodeAt(0) === 0xFEFF) {
    t = t.substring(1);
  }
  const raw: string[] = t.split('\n');
  const keep: string[] = [];
  for (let i = 0; i < raw.length; i++) {
    const line: string = raw[i].replace('\r', '');
    if (line.trimStart().startsWith('//')) {
      continue;
    }
    keep.push(line);
  }
  return keep.join('\n').replace(/"class"\s*:/g, '"catList":');
}
static parse(txt: string): JResp {
  const t: string = Cms.cleanJson(txt);
  if (t.trim().length === 0) {
    throw new Error('接口返回空内容');
  }
  try {
    return JSON.parse(t) as JResp;
  } catch (err) {
    throw new Error('接口返回的不是合法 JSON（数据源可能已失效）');
  }
}
static root(j: JResp): JResp {
  if (j.data !== undefined && j.data !== null) {
    return j.data;
  }
  return j;
}
/** HTML 实体 + 标签清理，用于简介 */
static plain(s: string): string {
  let r: string = s.replace(/<br\s*\/?>/gi, '\n');
  r = r.replace(/<\/p>/gi, '\n');
  r = r.replace(/<[^>]+>/g, '');
```

```
  r = r.replace(/&lt;/g, '<').replace(/&gt;/g, '>').replace(/&quot;/g, '"')
    .replace(/&#39;/g, "'").replace(/ /g, ' ').replace(/&amp;/g, '&');
  r = r.replace(/\n{3,}/g, '\n\n');
  return r.trim();
}
/** XML 节点取值：先取 CDATA 内容，再清标签与实体。
 *  注意顺序——不能先跑 plain()，否则 <![CDATA[片名]]> 会被当成标签整段删掉。 */
static node(block: string, tag: string): string {
  const re = new RegExp('<' + tag + '>([\\s\\S]*?)</' + tag + '>', 'i');
  const m = block.match(re);
  if (m === null) {
    return '';
  }
  return Cms.text(m[1]);
}
static text(s: string): string {
  let r: string = s;
  const c: number = r.indexOf('<![CDATA[');
  if (c >= 0) {
    const e: number = r.indexOf(']]>', c);
    if (e > 0) {
      r = r.substring(c + 9, e);
    }
  }
  return Cms.plain(r);
}
/** 部分站点的 ?ac=list 不返回 <class>，而是直接返回影片列表。
 *  这时从每条影片的 tid/type 反推分类，比"没有分类"更好用。 */
static catsFromVideos(xml: string): Category[] {
  const out: Category[] = [];
  const seen: string[] = [];
  const blocks = xml.match(/<video>[\s\S]*?<\/video>/gi);
  if (blocks === null) {
    return out;
  }
  for (let i = 0; i < blocks.length; i++) {
    const tid: string = Cms.node(blocks[i], 'tid');
    const tn: string = Cms.node(blocks[i], 'type');
    if (tid.length > 0 && tn.length > 0 && seen.indexOf(tid) < 0) {
      seen.push(tid);
      const c = new Category();
      c.id = tid;
      c.name = tn;
      out.push(c);
    }
  }
  return out;
}
static catFromXml(xml: string): Category[] {
  const out: Category[] = [];
```

```
  const re = /<ty\b[^>]*\bid="(\d+)"[^>]*>([\s\S]*?)<\/ty>/gi;
  let m = re.exec(xml);
  while (m !== null) {
    const c = new Category();
    c.id = m[1];
    c.name = Cms.plain(m[2]);
    if (c.name.length > 0) {
      out.push(c);
    }
    m = re.exec(xml);
  }
  return out;
}
static listFromXml(xml: string, page: number, site: Site): VodPage {
  const pg = new VodPage();
  pg.page = page;
  const head = xml.match(/<list\b[^>]*>/i);
  if (head !== null) {
    const p = head[0].match(/\bpage="(\d+)"/i);
    const pc = head[0].match(/\bpagecount="(\d+)"/i);
    const rc = head[0].match(/\brecordcount="(\d+)"/i);
    if (p !== null) {
      pg.page = parseInt(p[1], 10);
    }
    if (pc !== null) {
      pg.pageCount = parseInt(pc[1], 10);
    }
    if (rc !== null) {
      pg.total = parseInt(rc[1], 10);
    }
  }
  const blocks = xml.match(/<video>[\s\S]*?<\/video>/gi);
  if (blocks !== null) {
    for (let i = 0; i < blocks.length; i++) {
      const b: string = blocks[i];
      const v = new Vod();
      v.id = Cms.node(b, 'id');
      v.name = Cms.node(b, 'name');
      v.pic = Cms.node(b, 'pic');
      const note: string = Cms.node(b, 'note');
      v.remark = note.length > 0 ? note : Cms.node(b, 'dt');
      v.typeName = Cms.node(b, 'type');
      v.siteKey = site.key;
      v.siteName = site.name;
      if (v.name.length > 0) {
        pg.items.push(v);
      }
    }
  }
  return pg;
```

```
}
static detailFromXml(xml: string, site: Site, id: string): Vod {
  const blocks = xml.match(/<video>[\s\S]*?<\/video>/gi);
  if (blocks === null || blocks.length === 0) {
    throw new Error('接口没有返回详情数据');
  }
  const b: string = blocks[0];
  const v = new Vod();
  v.id = Cms.node(b, 'id');
  if (v.id.length === 0) {
    v.id = id;
  }
  v.name = Cms.node(b, 'name');
  v.pic = Cms.node(b, 'pic');
  v.remark = Cms.node(b, 'note');
  v.typeName = Cms.node(b, 'type');
  v.year = Cms.node(b, 'year');
  v.area = Cms.node(b, 'area');
  v.actor = Cms.node(b, 'actor');
  v.director = Cms.node(b, 'director');
  v.content = Cms.node(b, 'des');
  v.siteKey = site.key;
  v.siteName = site.name;
  // <dl><dd flag="ckm3u8">第1集$http://...</dd></dl>
  const flags: string[] = [];
  const urls: string[] = [];
  const ddRe = /<dd\b[^>]*\bflag="([^"]*)"[^>]*>([\s\S]*?)<\/dd>/gi;
  let m = ddRe.exec(b);
  while (m !== null) {
    flags.push(m[1]);
    urls.push(Cms.text(m[2]));
    m = ddRe.exec(b);
  }
  if (flags.length === 0) {
    const ddRe2 = /<dd\b[^>]*>([\s\S]*?)<\/dd>/gi;
    let m2 = ddRe2.exec(b);
    let n: number = 1;
    while (m2 !== null) {
      flags.push('线路' + n);
      urls.push(Cms.text(m2[1]));
      n++;
      m2 = ddRe2.exec(b);
    }
  }
  v.from = flags.join('$$$');
  v.playUrl = urls.join('$$$');
  return v;
}
/** vod_play_url / vod_play_from → 线路与剧集 */
static splitLines(from: string, playUrl: string): Line[] {
```

```
    const out: Line[] = [];
    if (playUrl.length === 0) {
      return out;
    }
    const fArr: string[] = from.length > 0 ? from.split('$$$') : [];
    const uArr: string[] = playUrl.split('$$$');
    for (let i = 0; i < uArr.length; i++) {
      const ln = new Line();
      ln.name = i < fArr.length && fArr[i].length > 0 ? fArr[i] : ('线路 ' + (i +
1));
      const segs: string[] = uArr[i].split('#');
      for (let k = 0; k < segs.length; k++) {
        const seg: string = segs[k].trim();
        if (seg.length === 0) {
          continue;
        }
        const ep = new Episode();
        const p: number = seg.indexOf('$');
        if (p > 0) {
          ep.name = seg.substring(0, p).trim();
          ep.url = seg.substring(p + 1).trim();
        } else {
          ep.name = '第 ' + (k + 1) + ' 集';
          ep.url = seg;
        }
        if (ep.url.length > 0 && ep.url.indexOf('://') > 0) {
          ln.eps.push(ep);
        }
      }
      // 标记这条线路能否直接喂给播放器：常见站点第一组是网页地址
      //  （如 https://cdn/play/xxx），第二组才是真流（.../index.m3u8）
      let directCount: number = 0;
      for (let k = 0; k < ln.eps.length; k++) {
        if (Cms.isDirect(ln.eps[k].url)) {
          directCount++;
        }
      }
      ln.direct = ln.eps.length > 0 && directCount * 2 >= ln.eps.length;
      if (ln.eps.length > 0) {
        out.push(ln);
      }
    }
    return out;
  }
  static isDirect(url: string): boolean {
    const u: string = url.toLowerCase();
    return u.indexOf('.m3u8') >= 0 || u.indexOf('.mp4') >= 0 || u.indexOf('.flv')
>= 0 ||
      u.indexOf('.mkv') >= 0 || u.indexOf('.ts') >= 0 || u.indexOf('.m4v') >= 0;
  }
```

```
  /**
   * 从站点 ext 里取出该分类的筛选组。
   * ext 形如 {"class":[...],"filters":{"1":[{"key":"area","name":"地区","value":[
{"n":"全部","v":""}]}]}}
   * 没有 ext、不是 JSON、或该分类没配筛选时返回空数组。
   */
  static filtersOf(ext: string, tid: string): FilterGroup[] {
    const out: FilterGroup[] = [];
    const txt: string = ext.trim();
    if (txt.length === 0 || txt.charAt(0) !== '{') {
      return out;
    }
    let j: JExt;
    try {
      j = JSON.parse(txt) as JExt;
    } catch (err) {
      return out;
    }
    const map: Record<string, JFilterGroup[]> | undefined = j.filters;
    if (map === undefined) {
      return out;
    }
    const groups: JFilterGroup[] | undefined = map[tid];
    if (groups === undefined || groups === null) {
      return out;
    }
    for (let i = 0; i < groups.length; i++) {
      const src: JFilterGroup = groups[i];
      const g = new FilterGroup();
      g.key = src.key === undefined ? '' : src.key;
      g.name = src.name === undefined ? g.key : src.name;
      const vs: JFilterValue[] | undefined = src.value;
      if (vs !== undefined) {
        for (let k = 0; k < vs.length; k++) {
          const sv: JFilterValue = vs[k];
          const v = new FilterValue();
          v.n = sv.n === undefined ? '' : sv.n;
          v.v = sv.v === undefined ? '' : sv.v;
          if (v.n.length > 0) {
            g.values.push(v);
          }
        }
      }
      if (g.key.length > 0 && g.values.length > 0) {
        out.push(g);
      }
    }
    return out;
  }
}
```

```
export function mapClasses(list: JClass[]): Category[] {
  const out: Category[] = [];
  for (let i = 0; i < list.length; i++) {
    const c = new Category();
    c.id = String(list[i].type_id === undefined ? '' : list[i].type_id);
    c.name = Cms.s(list[i].type_name);
    out.push(c);
  }
  return out;
}
// ============ 站点 ext 的原始结构（只取筛选那一段） ============
interface JFilterValue {
  n?: string;
  v?: string;
}
interface JFilterGroup {
  key?: string;
  name?: string;
  value?: JFilterValue[];
}
interface JExt {
  filters?: Record<string, JFilterGroup[]>;
}
// ================================================================================
// 文件：entry/src/main/ets/common/DataSource.ets     （共 49 行）
// ================================================================================
/**
 * LazyForEach 用的数组数据源。
 *
 * 为什么需要它：`ForEach` 会把**全部**数据一次性建成子组件，几百个频道就是几百个
 * GridItem（每个还带 Text、边框、动画），首屏耗内存也慢；`LazyForEach` 只创建
 * 可视区附近的（配合 Grid 的 cachedCount），滚动时边回收边建。
 *
 * 用法约定：整表替换一律走 `set()`（内部 onDataReloaded）。局部增删没必要——
 * 直播的列表变化（换分组/搜索/收藏/失效标记）本来就是成片变化的，逐条 notify
 * 只会让代码更难维护。
 */
export class ArraySource<T> implements IDataSource {
  private items: T[] = [];
  private listeners: DataChangeListener[] = [];
  /** 换一整批数据 */
  set(next: T[]): void {
    this.items = next;
    for (let i = 0; i < this.listeners.length; i++) {
      this.listeners[i].onDataReloaded();
    }
  }
  /** 当前数据长度（调用方判断空态用，不必再算一遍） */
  size(): number {
    return this.items.length;
```

```
  }
  totalCount(): number {
    return this.items.length;
  }
  getData(index: number): T {
    return this.items[index];
  }
  registerDataChangeListener(listener: DataChangeListener): void {
    if (this.listeners.indexOf(listener) < 0) {
      this.listeners.push(listener);
    }
  }
  unregisterDataChangeListener(listener: DataChangeListener): void {
    const i: number = this.listeners.indexOf(listener);
    if (i >= 0) {
      this.listeners.splice(i, 1);
    }
  }
}
// ================================================================================
// 文件：entry/src/main/ets/common/Device.ets     （共 90 行）
// ================================================================================
/**
 * 设备形态识别 —— 让同一套 UI 在「手机 / 电视」上各自适配。
 *
 * 判据：
 *   1. deviceInfo.deviceType 是官方分类，取值 'phone' | 'tv' | 'tablet' | '2in1'
| 'wearable' | 'default'。
 *      手机（含折叠屏 Pura X）返回 'phone'，电视返回 'tv'，这是主判据。
 *   2. 兜底：有些固件/模拟器返回 'default' / 'tablet'，此时用屏幕短边（vp）猜：
 *      短边 < 600vp 视为手机。电视横屏大屏、平板大屏都不落在这个区间。
 *
 * 结果写进 AppStorage（全局单例，跨页面共享）：
 *   isPhone : boolean  —— 手机布局（底部 Tab + 网格 + 触摸）
 *   isTv    : boolean  —— 电视布局（顶部功能行 + 横向轨道 + 遥控器焦点）
 *
 * 页面用 @StorageProp('isPhone') isPhone: boolean = false 读取。
 * Device.init() 必须在 loadContent 之前调用（放 EntryAbility.onCreate 最前面），
 * 这样首页第一次 build 就能读到正确值，不会闪一帧错误的布局。
 */
import deviceInfo from '@ohos.deviceInfo';
import display from '@ohos.display';
/** AppStorage 键名，页面用 @StorageProp 绑定 */
export const KEY_IS_PHONE: string = 'isPhone';
export const KEY_IS_TV: string = 'isTv';
export class Device {
  /** 入口初始化，必须在 EntryAbility.onCreate 里最先调用 */
  static init(): void {
    let dt: string = 'default';
    try {
```

```
    dt = deviceInfo.deviceType;
  } catch (e) {
    dt = 'default';
  }
  let isPhone: boolean = dt === 'phone';
  let isTv: boolean = dt === 'tv';
  // 官方分类拿不准时（default / tablet / 2in1 / wearable），用屏幕短边兜底
  if (dt !== 'phone' && dt !== 'tv') {
    try {
      const d = display.getDefaultDisplaySync();
      // 注意：display 的 width/height 单位是 **px**，不是 vp。
      // 原来直接拿 px 和 600 比，手机短边 1080px、平板 1600px 全都 > 600，
      // 兜底分支永远判不出手机 —— default/tablet/折叠屏 会落到电视布局
      //  （6 列网格 + 遥控焦点），在这些机器上完全没法用。
      // 一律先换成 vp 再比：手机 360~440vp，平板 600+vp。
      const shortVp: number = Math.min(px2vp(d.width), px2vp(d.height));
      if (shortVp > 0 && shortVp < 600) {
        isPhone = true;
        isTv = false;
      }
    } catch (e) {
      // 拿不到屏幕信息就维持默认（按电视处理）
    }
  }
  AppStorage.setOrCreate<boolean>(KEY_IS_PHONE, isPhone);
  AppStorage.setOrCreate<boolean>(KEY_IS_TV, isTv);
}
/** 是否是手机（含折叠屏）。UI 层优先用 @StorageProp 读，这里给非 UI 代码用 */
static isPhone(): boolean {
  return AppStorage.get<boolean>(KEY_IS_PHONE) === true;
}
/** 是否是电视 */
static isTv(): boolean {
  return AppStorage.get<boolean>(KEY_IS_TV) === true;
}
/**
 * 弹窗/浮层/面板宽度：手机竖屏用相对宽度留边（固定宽度会超出屏宽被裁切），
 * 电视端保持固定宽度不变。isPhone 在启动时已定、运行期不变，静态读取即可。
 */
static panelW(base: number): string | number {
  return Device.isPhone() ? '92%' : base;
}
/** 手机竖屏时留出状态栏/安全区后的可用宽度（vp）下限，供需要精确计算的场景用 */
static phoneW(): number {
  try {
    const d = display.getDefaultDisplaySync();
    // 同样要换单位：返回值是 px，而调用方按 vp 用来算布局
    return Math.min(px2vp(d.width), px2vp(d.height));
  } catch (e) {
    return 440;
```

```
    }
  }
}
// ============================================================================
// 文件：entry/src/main/ets/common/Feedback.ets     （共 105 行）
// ============================================================================
/**
 * 全局点击反馈（声音 + 震动）。
 *
 * 用户按任意按钮时给一个「声音 + 震动」的即时反馈，让操作有实感。
 * 两个通道各自独立开关，偏好存本机 preferences：
 *   tapSound   0 = 关 / 1 = 开（默认开）
 *   tapVibrate 0 = 关 / 1 = 开（默认开）
 *
 * 声音用 SoundPool（低延迟短音效，播放 28ms 的 click.wav，混音模式不打断正在播的
片），
 * 震动用 vibrator 触发一次 20ms 短震。二者都 fire-and-forget，
 * 不阻塞 UI 线程，点击响应依然跟手。
 *
 * 用法（任何页面）：
 *   import { Feedback } from '../common/Feedback';
 *   .onClick(() => { Feedback.tap(); this.xxx(); })
 *
 * 启动时 EntryAbility.onCreate 调 Feedback.init(this.context) 加载音效；
 * 设置页切换开关后调 Feedback.refresh() 刷新内存缓存。
 */
import vibrator from '@ohos.vibrator';
import soundPool from '@ohos.multimedia.soundPool';
import audio from '@ohos.multimedia.audio';
import common from '@ohos.app.ability.common';
import { Store } from './Store';
export class Feedback {
  private static sp: soundPool.SoundPool | null = null;
  private static soundId: number = -1;
  private static ready: boolean = false;
  private static soundOn: boolean = true;
  private static vibrateOn: boolean = true;
  private static inited: boolean = false;
  /** 启动时调用一次（EntryAbility.onCreate），传入 context 用于加载 rawfile 音效
*/
  static init(ctx?: common.UIAbilityContext): void {
    if (Feedback.inited) {
      return;
    }
    Feedback.inited = true;
    Feedback.refresh();
    if (ctx !== undefined) {
      Feedback.loadSound(ctx);
    }
  }
```

```
  private static loadSound(ctx: common.UIAbilityContext): void {
    try {
      const info: audio.AudioRendererInfo = {
        usage: audio.StreamUsage.STREAM_USAGE_MUSIC,
        rendererFlags: 0
      };
      Feedback.sp = soundPool.createSoundPool(1, info);
      ctx.resourceManager.getRawFd('click.wav').then((fd) => {
        if (Feedback.sp === null) {
          return;
        }
        Feedback.sp.load(fd.fd, fd.offset, fd.length).then((id: number) => {
          Feedback.soundId = id;
          Feedback.ready = true;
        }).catch(() => {
        });
      }).catch(() => {
      });
    } catch (e) {
      // 音效加载失败不影响震动反馈
    }
  }
  /** 开关变更后刷新内存缓存（设置页切换开关时调用） */
  static refresh(): void {
    Store.tapSound().then((v: number) => {
      Feedback.soundOn = v !== 0;
    });
    Store.tapVibrate().then((v: number) => {
      Feedback.vibrateOn = v !== 0;
    });
  }
  /** 点一下的反馈（任何按钮 onClick 开头调用） */
  static tap(): void {
    if (Feedback.vibrateOn) {
      try {
        vibrator.vibrate(20);
      } catch (e) {
        // 忽略：个别设备无马达或权限受限
      }
    }
    if (Feedback.soundOn && Feedback.ready && Feedback.sp !== null && Feedback.sou
ndId >= 0) {
      try {
        const params: soundPool.PlayParameters = {
          loop: 0,
          rate: 1,
          leftVolume: 0.7,
          rightVolume: 0.7,
          priority: 0
        };
```

```
        Feedback.sp.play(Feedback.soundId, params).catch(() => {
        });
      } catch (e) {
        // 忽略
      }
    }
  }
}
// ============================================================================
// 文件：entry/src/main/ets/common/LiveProbe.ets     （共 228 行）
// ============================================================================
/**
 * 播放源测速排行 + 可用性检测。
 *
 * 为什么抽成公共模块：这套逻辑原先只写在设置页里，直播页拿不到，于是用户在
 * 直播页看到某个源「一堆台但全点不开」，只能自己退回设置页去猜哪个源好用。
 * 现在两处共用同一份实现，结论也一致（同一套超时、抽样数、评分权重）。
 *
 * 两类检测，回答的是两个不同问题：
 *   1. 源级 rankOne —— 这个源好不好？拉列表快不快、有多少台、抽样能不能播。
 *      用于「测速排行」，用来在多个源之间挑一个。
 *   2. 频道级 channels —— 这个源里哪些台是死的？逐个抓首包，
 *      用于把失效台标灰，避免用户点一次等 20 秒。
 *
 * 一个必须记住的教训：只看「列表拉得快」完全没用。实测有源 74 ms 就拉完
 * 500 多个台，抽样一测全连不上。所以可播率必须实测，不能靠速度推断。
 */
import { Live } from './Live';
import { Channel, ChannelGroup } from './Model';
/** 一次源级测速的结论 */
export class LiveScore {
  name: string = '';
  url: string = '';
  /** 列表是否拉得下来（不代表频道能播） */
  ok: boolean = false;
  /** 拉完整份列表的耗时（ms），-1 = 没测出来 */
  ms: number = -1;
  /** 频道总数 */
  count: number = 0;
  /** 抽样里能连上的数量 */
  alive: number = 0;
  /** 实际抽样数量（源里台太少时可能小于计划值） */
  sampled: number = 0;
  /** 综合评分 0~100，越大越好 */
  score: number = 0;
  /** 给界面直接用的一句话，避免每处各拼一遍 */
  state: string = '';
}
export class LiveProbe {
  /** 每个源抽样几个台实测。3 个够看出「全军覆没」，再多会明显拖慢排行 */
```

```
  static readonly SAMPLE: number = 3;
  /** 单条流等首包的超时。死链常常直接挂住不返回，超过这个时间就当失效 */
  static readonly STREAM_MS: number = 3000;
  /** 拉列表的超时。8 秒拉不下来说明这个源本身有问题 */
  static readonly LIST_MS: number = 8000;
  /** 外层兜底超时：Http 的 abort 不是所有情况都能打断，外层再压一道 */
  static readonly GUARD_MS: number = 10000;
  /**
   * 测一个源。不会抛异常，失败也会返回一个 ok=false 的结果。
   *
   * 顺序是刻意的：先把列表拉下来（拿到频道数），再抽样测流。
   * 反过来做的话，列表都拉不下来的源还要白等 3 次流超时。
   */
  static async rankOne(name: string, url: string): Promise<LiveScore> {
    const r = new LiveScore();
    r.name = name;
    r.url = url;
    const t0: number = Date.now();
    let gs: ChannelGroup[] = [];
    try {
      gs = await Promise.race([Live.load(url, LiveProbe.LIST_MS), LiveProbe.guard(
LiveProbe.GUARD_MS)]);
    } catch (e) {
      r.state = '拉取失败';
      return r;
    }
    r.ms = Date.now() - t0;
    let n: number = 0;
    for (let i = 0; i < gs.length; i++) {
      n += gs[i].channels.length;
    }
    if (n === 0) {
      r.state = '列表为空';
      return r;
    }
    r.ok = true;
    r.count = n;
    // 抽样：先从不同分组里各取第一个（避免全落在同一个台的多条线路上），
    // 分组数不够再回头补该组里的第 2、3 个 —— 有的源只有一个分组装着四百个台，
    // 只从这里取一个样本，「可播 1/1」的说服力太弱，一个台抽风就会把整个源判死。
    const samples: string[] = [];
    for (let i = 0; i < gs.length && samples.length < LiveProbe.SAMPLE; i++) {
      const chs: Channel[] = gs[i].channels;
      if (chs.length > 0) {
        samples.push(chs[0].urls.length > 0 ? chs[0].urls[0] : chs[0].url);
      }
    }
    for (let k = 1; samples.length < LiveProbe.SAMPLE; k++) {
      let added: boolean = false;
      for (let i = 0; i < gs.length && samples.length < LiveProbe.SAMPLE; i++) {
```

```
        const chs: Channel[] = gs[i].channels;
        if (k < chs.length) {
          const u: string = chs[k].urls.length > 0 ? chs[k].urls[0] : chs[k].url;
          if (samples.indexOf(u) < 0) {
            samples.push(u);
            added = true;
          }
        }
      }
      if (!added) {
        break;      // 全源就那么几个台，补不出来
      }
    }
    r.sampled = samples.length;
    const jobs: Promise<boolean>[] = [];
    for (let i = 0; i < samples.length; i++) {
      jobs.push(Live.probe(samples[i], LiveProbe.STREAM_MS));
    }
    const rs: boolean[] = await Promise.all(jobs);
    for (let i = 0; i < rs.length; i++) {
      if (rs[i]) {
        r.alive++;
      }
    }
    r.score = LiveProbe.score(r);
    r.state = r.ms + ' ms · ' + r.count + ' 台 · 可播 ' + r.alive + '/' + r.sample
d;
    return r;
  }
  /**
   * 评分：可播率占 70%，速度占 30%。
   *
   * 权重不对称是有意的 —— 直播首要的是「点开能看」。一个 200 ms 但半数是死台的源
,
   * 实际体验远不如一个 800 ms 但全部能播的源。速度按 50 ms 折 1 分递减：
   * 100 ms → 98 分，1000 ms → 80 分，5000 ms → 0 分。
   */
  static score(r: LiveScore): number {
    if (!r.ok) {
      return 0;
    }
    const availability: number = r.sampled > 0 ? (r.alive / r.sampled) * 100 : 0;
    const speed: number = Math.max(0, 100 - r.ms / 50);
    return Math.round(availability * 0.7 + speed * 0.3);
  }
  /**
   * 批量测速，按批推进（每批 3 个）。
   *
   * 并发全部拉满的话，同时开十几个 HTTP 连接，弱网下会互相拖慢，
   * 测出来的耗时反而不准（本来 300 ms 的源测成 2 秒）。
```

```
   */
  static async rankAll(items: LiveScore[], onEach: (i: number, r: LiveScore) => vo
id): Promise<LiveScore[]> {
    const out: LiveScore[] = [];
    for (let i = 0; i < items.length; i++) {
      const r = new LiveScore();
      r.name = items[i].name;
      r.url = items[i].url;
      out.push(r);
    }
    for (let s = 0; s < out.length; s += 3) {
      const batch: Promise<void>[] = [];
      for (let i = s; i < Math.min(s + 3, out.length); i++) {
        const idx: number = i;
        batch.push((async (): Promise<void> => {
          const r = await LiveProbe.rankOne(out[idx].name, out[idx].url);
          out[idx] = r;
          onEach(idx, r);
        })());
      }
      await Promise.all(batch);
    }
    return out;
  }
  /** 按评分排序；同分时台多的排前面，再同则按耗时 */
  static sort(list: LiveScore[]): LiveScore[] {
    const out: LiveScore[] = list.slice();
    out.sort((a: LiveScore, b: LiveScore): number => {
      if (a.score !== b.score) {
        return b.score - a.score;
      }
      if (a.ok !== b.ok) {
        return a.ok ? -1 : 1;
      }
      if (a.count !== b.count) {
        return b.count - a.count;
      }
      return a.ms - b.ms;
    });
    return out;
  }
  /**
   * 逐个检测频道能不能连上，返回失效数量。
   *
   * 并发控制在 4：再高容易把源站当攻击，几十个台同时探活会被限流，
   * 结果是一堆「假失效」——明明能播的台被标灰，比不标更糟。
   *
   * onTick 每检测完一条调一次，界面可以边测边标灰，不用等全部结束。
   */
  static async channels(chs: Channel[], onTick: (ch: Channel, alive: boolean) => v
```

```
oid,
    parallel: number = 4): Promise<number> {
    let dead: number = 0;
    for (let s = 0; s < chs.length; s += parallel) {
      const jobs: Promise<void>[] = [];
      for (let i = s; i < Math.min(s + parallel, chs.length); i++) {
        const ch: Channel = chs[i];
        const u: string = ch.urls.length > 0 ? ch.urls[0] : ch.url;
        jobs.push((async (): Promise<void> => {
          const ok: boolean = await Live.probe(u, LiveProbe.STREAM_MS);
          if (!ok) {
            dead++;
          }
          onTick(ch, ok);
        })());
      }
      await Promise.all(jobs);
    }
    return dead;
  }
  /** 外层兜底超时：到点就抛，让 rankOne 走到「拉取失败」分支 */
  private static guard(ms: number): Promise<ChannelGroup[]> {
    return new Promise<ChannelGroup[]>((_, reject: (e: Error) => void) => {
      setTimeout(() => {
        reject(new Error('探测超时'));
      }, ms);
    });
  }
}
// ============================================================================
// 文件：entry/src/main/ets/common/Net.ets     （共 108 行）
// ============================================================================
/**
 * 网络状态：当前能不能出网 + 断网/恢复时给一次轻提示。
 *
 * 为什么不做成每个页面各自弹「网络错误」：一个断网动作会让首页、直播、搜索
 * 同时失败，用户一次看到三处红字。这里统一在系统层监听，状态翻转时只提示
 * 一次（Tip 是全局的），页面只管照常显示自己的错误文案。
 *
 * 判定优先用 NET_CAPABILITY_VALIDATED 而不是「有没有连上 WiFi」：
 * 连上路由器但没通外网（酒店门户、路由器没拨号）时 VALIDATED 不带，
 * 这时提示「已连接 WiFi」是错的，用户会一直以为是应用本身的问题。
 */
import connection from '@ohos.net.connection';
import hilog from '@ohos.hilog';
import { BusinessError } from '@ohos.base';
import { Tip } from './Tip';
/** AppStorage 键：'1' 在线 / '0' 离线。页面可选读它来改文案 */
export const NET_ONLINE: string = 'netOnline';
export class Net {
```

```
  private static inited: boolean = false;
  /** 上一次的在线状态，用来只在翻转时提示 */
  private static lastOnline: boolean = true;
  /** 监听句柄，必须留着才能 unregister（也防止被 GC 后监听失效） */
  private static nc: connection.NetConnection | null = null;
  /** 进程内初始化一次：登记网络变化监听 + 先判一次当前状态 */
  static init(): void {
    if (Net.inited) {
      return;
    }
    Net.inited = true;
    AppStorage.setOrCreate(NET_ONLINE, '1');
    try {
      const nc: connection.NetConnection = connection.createNetConnection();
      // 四个事件都要收：只监听 netLost 会漏掉「WiFi 还连着但出不了网」
      // 这种最常见的情况（拔网线、路由器断网），那类变化走 netCapabilitiesChange
      nc.on('netAvailable', () => {
        Net.apply();
      });
      nc.on('netLost', () => {
        Net.apply();
      });
      nc.on('netCapabilitiesChange', () => {
        Net.apply();
      });
      nc.on('netUnavailable', () => {
        Net.apply();
      });
      nc.register((err: BusinessError) => {
        if (err) {
          hilog.info(0x1000, 'BOXN', 'net register failed code=%{public}d', err.co
de);
        }
      });
      Net.nc = nc;
    } catch (e) {
      // 没有 GET_NETWORK_INFO 权限或系统不支持：退化成「只在请求失败时判定」
      hilog.info(0x1000, 'BOXN', 'net watch unavailable, fallback to passive check
');
    }
    Net.apply();
    hilog.info(0x1000, 'BOXN', 'net init online=%{public}s', Net.online() ? 'y' :
'n');
  }
  /**
   * 现在能不能出网。
   * 取不到能力集时一律返回 true —— 宁可漏报，也不要因为一次系统调用异常
   * 就把用户正在用的网络判成「断网」，那会让所有页面都挂上错误态。
   */
  static online(): boolean {
```

```
    try {
      const h: connection.NetHandle = connection.getDefaultNetSync();
      if (h === null || h.netId === 0) {
        return false;
      }
      const caps: connection.NetCapabilities = connection.getNetCapabilitiesSync(h
);
      const list: connection.NetCap[] | undefined = caps.networkCap;
      if (list === undefined) {
        // 老版本系统不返回能力集，只能认为「有默认网络就算在线」
        return true;
      }
      if (list.indexOf(connection.NetCap.NET_CAPABILITY_VALIDATED) >= 0) {
        return true;
      }
      // 没有 VALIDATED 但声明了 INTERNET：可能只是联通性探测还没出结果，
      // 判成离线会误伤刚开始播放的用户，交给真实请求去证伪
      return list.indexOf(connection.NetCap.NET_CAPABILITY_INTERNET) >= 0;
    } catch (e) {
      return true;
    }
  }
  /** 判一次并处理状态翻转（提示 + 写 AppStorage） */
  private static apply(): void {
    const on: boolean = Net.online();
    AppStorage.setOrCreate(NET_ONLINE, on ? '1' : '0');
    if (on === Net.lastOnline) {
      return;
    }
    Net.lastOnline = on;
    hilog.info(0x1000, 'BOXN', 'net online=%{public}s', on ? 'y' : 'n');
    if (on) {
      Tip.show('网络已恢复');
    } else {
      Tip.show('网络已断开，请检查 WiFi 或网线');
    }
  }
}
// ======================================================================
// 文件：entry/src/main/ets/common/PhoneTabBar.ets     （共 77 行）
// ======================================================================
/**
 * 手机端底部 Tab 导航 —— 全局共享，供首页和各子页面复用。
 *
 * 电视端没有这个：电视用顶部功能行 + 遥控器焦点导航。
 * 手机端首页和各功能页底部都放同一个栏，用户不用回首页就能切换页面。
 *
 * active 传「当前页面的路由」，对应项高亮、点击不重复跳转（避免页面无限堆叠）；
 * 首页传空字符串（首页不属于任何子页面 Tab）。
 */
```

```
import router from '@ohos.router';
import { T } from './Model';
function tabIcon(label: string): Resource {
  if (label === '直播') {
    return $r('app.media.ic_live');
  }
  if (label === '搜索') {
    return $r('app.media.ic_search');
  }
  if (label === '收藏') {
    return $r('app.media.ic_collect');
  }
  if (label === '设置') {
    return $r('app.media.ic_setting');
  }
  return $r('app.media.ic_vod');
}
/** 单个 Tab 项：图标 + 文字，整块可点 */
@Builder
function TabItem(label: string, page: string, active: string) {
  Column() {
    Image(tabIcon(label))
      .width(20)
      .height(20)
      .objectFit(ImageFit.Contain)
      // 白色单色 svg：浅色主题下要染色，否则图标消失（只剩文字）
      .fillColor(page === active ? T.SELECTED : T.SUB)
    Text(label)
      .fontSize(11)
      .fontColor(page === active ? T.SELECTED : T.SUB)
      .margin({ top: 3 })
      .maxLines(1)
  }
  .layoutWeight(1)
  .height('100%')
  .justifyContent(FlexAlign.Center)
  .onClick(() => {
    if (page !== active) {
      // 从首页进子页用 push（保留首页在栈底，back 能回首页）；
      // 子页之间切换用 replace（替换当前页，来回点 tab 不堆栈，back 直接回首页）
。
      if (active === '') {
        router.pushUrl({ url: page });
      } else {
        router.replaceUrl({ url: page });
      }
    }
  })
}
/** 底部 Tab 栏：5 个入口，触屏拇指热区 */
```

```
@Builder
export function PhoneTabBar(active: string) {
  Row() {
    TabItem('点播', 'pages/VodPage', active)
    TabItem('直播', 'pages/LivePage', active)
    TabItem('搜索', 'pages/Search', active)
    TabItem('收藏', 'pages/CollectPage', active)
    TabItem('设置', 'pages/SettingPage', active)
  }
  .width('100%')
  .height(54)
  .padding({ left: 4, right: 4 })
  .backgroundColor(T.SURFACE)
  .border({ width: { top: 1 }, color: '#1FFFFFFF' })
}
// ================================================================================
// 文件：entry/src/main/ets/common/Pinyin.ets     （共 140 行）
// ================================================================================
/**
 * 汉字 → 拼音首字母（声母）映射，供搜索联想做「拼音首字母」猜测。
 *
 * 数据来源：GB2312 字库（一、二级共 6763 个汉字），用 macOS
 * CFStringTransform 批量转拼音后取首字母；另对 21 个影视语境下
 * 首字母易分歧的多音字（长/重/传/乐/行…）做了双声母修正，
 * 生成缩写时这类字会分叉出多个候选，保证「cz」也能命中「长征」。
 *
 * 说明：这是本地纯内存查表，不联网；表外字（超 GB2312 的生僻字）
 * 会被跳过，剩余字仍能提供足够首字母用于匹配。
 */
const MAIN: string = 'a:啊阿埃挨哎唉哀皑癌蔼矮艾碍爱隘鞍氨安俺按暗岸胺案肮昂盎凹敖
熬翱袄傲奥懊澳谙坳埯拗捱掊吖嗷嗳嗌岙犴庵廒遨媪媛鸯瑗桉葵暧砹铵锕锇鹌聱螯霭鏊鳌鏖黯
;b:芭捌扒叭吧笆八疤巴拔跋靶把耙坝霸罢爸白柏百摆佰败拜稗斑班搬扳般颁板版扮拌伴瓣半
办绊邦帮梆榜膀绑棒磅蚌镑傍谤苞胞包褒剥薄雹保堡饱宝抱报暴豹鲍爆杯碑悲卑北辈背贝钡倍
狈备惫焙被奔苯本笨崩绷甭泵蹦迸逼鼻比鄙笔彼碧蓖蔽毕毙毖币庇痹闭敝弊必壁臂避陛鞭边编
贬扁便变卞辨辩辫遍标彪膘表鳖憋别瘪彬斌濒滨宾摈兵冰柄丙秉饼炳病并玻菠播拨钵波博勃搏
铂箔伯帛舶脖膊渤驳捕卜哺补埠不布步簿部怖埔匕孝匾俾傧勹亳禀冫阪邢邶弁畚坌坂芘芣芟苹
荸菠萝葆蓓蒌蕃薛捭摒卟吡呗哔啵嘣岜豳猃饽庳忭悖愎汴浜浐濞灞逋弼妣婊婢嬖孢骠缏缤玢璧
杓槟檗殡瓿鬓睇贲掰擘膑踫飙飚煲煸灬砭碚碥碡畀钸钣钯钸铋锌镔镖镳秕鸨鹐鹌瘢瘭瘴窦裯
裨裨褓褊蝙笆筚箕篦簸舭舨襞粑醭蹩趵跋跸踣踣龅鏊鲅鳊鳔鞴髀髌魅髡鬓;c:擦猜裁材才财睬踩
采彩菜蔡餐参蚕残惭惨灿苍舱仓沧操糙槽曹草厕策侧册测层蹭插叉茬茶查碴搽察岔差诧拆柴豺
搀掺蝉馋谗缠铲产阐颤昌猖场尝常偿肠厂敞畅唱倡超抄钞嘲潮巢吵炒车扯撤掣彻澈郴臣辰尘晨
忱沉陈趁衬撑称城橙成呈乘程惩澄诚承逞骋秤吃痴持池迟弛驰耻齿侈尺赤翅斥炽充冲虫崇宠抽
酬畴踌稠愁筹仇绸瞅丑臭初出橱厨躇锄雏滁除楚础储矗搐触处揣川穿椽船喘串疮窗幢床闯创吹
炊捶锤垂春椿醇唇淳纯蠢戳绰疵茨磁雌辞慈瓷词此刺赐次聪葱囱匆从丛凑粗醋簇促蹿篡窜摧崔
催脆瘁粹淬翠村存寸磋撮搓措挫错伺畜于丞厝伥伧侪俦傺傺汆冁谄谌谶陲刍鬯坼埕墀艹茬苁茬茈
苋莼菖萃蒇蔟抻搋撺叱呲唬啐啜啻嗔嗤嘈嚓帱岑嵯彳徂徜猝猹舛饬馇廛忖忏忡怅怆怵怊恻恼
惆悴憧憷阊汊涔淙漕潺澶宸讠遄孱孱屮姹娼嫦嫦骖骢骡缁巛琮琛璀璁璨权杵枨枞柽楮棰槌榇
楛榱樗檫殂辍辏昶晁觇毳氅氚敕脞腠塍膪焯爨祠禅砗碜磉眵瞠钏钗铖铳锉锸镡镩镲矬鸱鹑鹆
痤瘥瘛瘳衩裎褚褫皴耖蚤蚩蛏蜍蝽螭蟾蟾笞篾舂舡艚艟粲糍豉醒趱蹙踔踹蹉蹶蹴躔龊龌雠鲳
魑黜黢;d:搭达答瘩打大呆歹傣戴带殆代贷袋待逮怠耽担丹郸掸胆旦氮但惮淡诞弹蛋当挡党荡
```

档刀捣蹈倒岛祷导到稻悼道盗德得的蹬灯登等瞪凳邓堤低滴迪敌笛狄涤翟嫡抵底地蒂第帝弟递缔颠掂滇碘点典靛垫电佃甸店惦奠淀殿碉叼雕凋刁掉吊钓调跌爹碟蝶迭谍叠丁盯叮钉顶鼎锭定订丢东冬董懂动栋侗恫冻洞兜抖斗陡豆逗痘都督毒犊独读堵睹赌杜镀肚度渡妒端短锻段断缎堆兑队对墩吨蹲敦顿囤钝盾遁掇哆多夺垛躲朵跺舵剁惰堕氐刂仃儋籴诋谛谎阽邸凶坫坻垭垌逮堞芏荻莒菪兢葺揲甙叨呔咚咄哒哚啖啶喋嗒嘀嗓嘀噔崇岱峒嶝巅忉怛沌渎澹宕迨妲娣驺绐缍玎玷玳柢棳棣椴殚戟赕觇氖氡牍牒胨胴脞炖煅怒憨沓砀砘砥硐碓碇碲礅噔镫眙眈睇铗钿铎铛铞铖铙镝镩镫赕鸹疔疸瘅癜癞窦裆裰褡耋盯聃蚪蠹笃笪箪筢簪簟筌羝蠹酊亟踮蹀踱貂鲷鲽靼鞑骶髑黛黩;e:蛾峨鹅俄额讹娥恶厄扼遏鄂饿恩而儿耳尔饵洱二贰噩诶诿垩茫莪萼蒽摁呃愕迤嗣婀珥轭腭铒锇铹鸸鹗颚鲃鳄;f:发罚筏伐乏阀法珐藩帆番翻樊矾钒繁凡烦反返范贩犯饭泛坊芳方肪房防妨仿访纺放菲非啡飞肥匪诽吠肺废沸费芬酚吩氛分纷坟焚汾粉奋份忿愤粪丰封枫蜂峰锋风疯烽逢冯缝讽奉凤佛否夫敷肤孵扶拂辐幅氟符伏俘服浮涪福袱弗甫抚辅俯釜斧腑府腐赴副覆赋复傅付阜父腹负富讣附妇缚咐匸俸偾匐凫阝邡郭酆垒芙苇苻茯荸菔葜蘩拊呋吼哝幙幡狒怫悱沣淝淦灨艉妃孚驸绂绋绯枋梵桴梦椭赙腓斐燔扉袚砜砧砩黢黼畈罘钫镄稃馥痱蚨蜉蜇蝠蝮缶筐舫翡麸趺跗蹯霏鲋鲋鲱鳢鳆鼢;g:噶嘎该改概钙盖溉干甘杆柑竿肝赶感秆敢赣冈刚钢缸肛纲岗港杠篙皋高膏羔糕搞镐稿告哥歌搁戈鸽胳疙割革葛格阁隔铬个各给根跟耕更庚羹埂耿梗工攻功恭龚供躬公宫弓巩汞拱贡共钩勾沟苟狗垢构购够辜菇咕箍估沽孤姑鼓古蛊骨谷股故顾固雇刮瓜剐寡挂褂乖拐怪棺关官冠观管馆罐惯灌贯光广逛瑰规圭硅归龟闺轨鬼诡癸桂柜跪贵刽辊滚棍锅郭国果裹过咯丐亘鬲丨辜嘏甌卦剁仡佝倌鲠衮诂诖诟诰陔郜哿圪坩垓涡塌苷茛莞菰藁廾尬尴捆掼擀吊呱咴哏哽嗝帼峋崮嶂犷猓庋赓汨泔淦渲澉宄遘妫媾忿叟绀绠绲缑缟珙枸桄栝桧桔椁槔槠橄轱豁旮旰杲晷赅觏牯牲搿虢肱胍膈臌彀榖戇斫硌磙瞽罡罟盥钆钴锆锢镉皈鸪鸹鹆鹳疳瘤袼聒蚣蚪蛄蜾蝈笱箕篝簋舸艮酐酤觚鲅鲑鲠鲧鲴鳏鳐鞴鹘骼;h:蛤哈骸孩海氦亥害骇酣憨邯韩含涵寒函喊罕翰撼捍旱憾悍焊汗汉夯杭航壕嚎豪毫郝好耗号浩呵喝荷菏核禾和何合盒貉阂河涸赫褐鹤贺嘿黑痕很狠恨哼亨横衡恒轰哄烘虹鸿洪宏弘红喉侯猴吼厚候后呼乎忽瑚壶葫胡蝴狐糊湖弧虎唬护互沪户花哗华猾滑画划化话槐徊怀淮坏欢环桓缓换患唤痪豢焕涣宦幻荒慌黄磺蝗簧皇凰惶煌晃幌恍谎灰挥辉徽恢蛔回毁悔慧卉惠晦贿秽会烩汇讳诲绘荤昏婚魂浑混豁活伙火获或惑霍货祸咳剿黉訇沍讧诃诙诨锃隍邗郸奂劾壑堠茴荟荭葜菡蒿蕙蘸薨薅藿蘅撖擐攉哎哕嗯喙嗬嗨嘻嚄囫圜岵後徨猢獾夥馁怙惚隙闵阊阖沆泓洹洄洽浒浣湟溷潢滹濠瀚灏寰逅逭遑彗骅纡绗缋缳琦琥璜桀桁榭轷昊曷晫晗肓胲觳烀焓煳庠扈祜恚耆盍钬铓铪锶锾镬皓瓠鹕鳝瘊癀耘顸颃颌颔颢卢虺蚌蚶蚴蟥蟪蠖笏篁篌糇翮醐醢踝斛蝗鲿鳇骺鬟麾鼾;j:击圾基机畸稽积箕肌饥迹激讥鸡姬绩缉吉极棘辑籍集及急疾汲即嫉级挤几脊己蓟技冀季伎祭剂悸济寄寂计记既忌际妓继纪嘉枷夹佳家加荚颊贾甲钾假稼价架驾嫁歼监坚尖笺间煎兼肩艰奸缄茧检柬碱硷拣捡简俭剪减荐鉴践贱见键箭件健舰剑饯渐溅涧建僵姜将浆江疆蒋桨奖讲匠酱蕉椒礁焦胶交郊浇骄娇嚼搅铰矫侥脚狡角饺缴绞剿教酵轿较叫窖揭接皆秸街阶截劫节桔杰捷睫竭洁结姐戒藉芥界借介疥诫届巾筋斤金今津襟紧锦仅谨进靳晋禁近烬浸尽劲荆兢茎睛晶鲸京惊精粳经井警景颈静境敬镜径痉靖竟竞净炯窘揪究纠玖韭久灸九酒厩救旧臼舅咎就疚鞠拘狙疽居驹菊局咀矩举沮聚拒据巨具距踞锯俱句惧炬剧捐鹃娟倦眷卷绢撅攫抉掘倔爵觉决诀绝均菌钧军君峻俊竣浚郡骏茄丌亟乩厥刭刽劂冂伽佶佼倨偈做僭僬傲讦讵诂谏谪谲卩卺阱邦郸矍鳘艽芟荽苣苴莒荞茭茳恝菁堇菅葭蒺蒹蕨蓑拮挢捃掎搛叽咭哜唧喈啾嗟嚯噍噤囝岌岬峤崛嵴徼狷獐獗馑廛憬阄泾浃洎洚涓湫湔蹇謇迥迦迳遽刁屐屣弨妗姣婧婕孑孓骥绛缙缣缦畿玑珏珈琚瑾枧枢桕桀桀桷楗椐楫榘檫樯橘殛戋戛戟戬炅贶赍觊鞒垒犏犍毽敫胼胛胫腈腱飓蔹旌爝扃恕矶碣礓睑羁蠲钅钜铗锏锔锇镌镓镢嵇稷皎鸠鹣鹪鹫疖痂瘕瘠窭衿裨裾皱矜耩颉虮蚴蛱蛟笈笄笕笳裟羯糨暨翦赳趄豆醮酾跰跚跻跤踽踺踽蹶觫觯靓霁龅隽睢鲔鲶鲛鲣鲫鞯鞫骯髻颢麂麋;k:槛喀咖卡开揩楷凯慨刊堪勘坎砍看康慷糠扛抗亢炕考拷烤靠坷苛柯棵磕颗科壳可渴克刻客课肯啃垦恳坑吭空恐孔控抠口扣寇枯哭窟苦酷库裤夸垮挎跨胯块筷侩快宽款匡筐狂框矿眶旷况亏盔岿窥葵奎魁傀馈愧溃坤昆捆困括扩廓阔垣匮刽凯蒯伉伟佟侃侥倥傀俩偌悰愕逛诓邵圹垲堀茏苡菖蒽蔻氚揆叩咔哐哙喱嗜嗌嗑崆狯恹恺恪悝悃愦闶阃阚溘逵尻骒纩绔缮珂琨栲轲戡暌观犒氪胨脍龛眍瞵瞌瞰钪钶铐铠铿锎锞锟稞疴窠裉聩颈蛞蝰蝌筘笆篑醌跬鲣骷髁髋髡;l:垃拉喇蜡腊辣啦莱来赖蓝婪栏拦篮阑兰澜谰揽览懒缆烂滥琅榔狼廊郎朗浪捞劳牢老佬姥酪烙涝勒雷镭蕾磊累儡垒擂肋类泪棱楞冷厘梨犁黎篱狸离漓理李里鲤礼莉荔吏栗丽厉励砾历利傈例俐痢立粒沥隶力璃哩俩联莲连镰廉怜涟帘敛脸链恋炼练粮凉梁粱良两辆量晾亮谅撩聊僚疗燎寥辽潦了撂镣廖料列裂烈劣猎琳林磷霖临邻鳞淋凛赁吝拎玲菱零龄铃

伶羚凌灵陵岭领另令溜琉榴硫馏留刘瘤流柳六龙聋咙笼窿隆垄拢陇楼娄搂篓漏陋芦卢颅庐炉掳卤虏鲁麓碌露路赂鹿潞禄录陆戮驴吕铝侣旅履屡缕虑氯律率滤绿峦挛孪滦卵乱掠略抡轮伦仑沦纶论萝螺罗逻锣箩骡裸落洛骆络剌仂俪俚倮偻裔赢羸冽诔丽酃坂咙垆坪塄塄栳茏茏荤荏茛蒌蓠潼蔹蔺蓼藜食旭捋捩摞撸叻呖呤咧唠啉啷唳喽嘞嘹噜囵囡岚崂峁嵝嶙徕猢猡僚廪愣懔闾阆泖泷泸泠泺洌浏涞渌溧溻漯潋澉澹濑澧濂寮逦逯遛遴邋娈娌蒌嫘骊骝绫缭缥缭珑珞琏璐栎栊栌栉栎栾棣椤棕楝榄桐橹檑檩殓铲轱辂辘辚瓠晃赍毯胧胪胤箐臁膦斓旒熘戾砺砻砬睐瞵詈嚣罹钌铑铹铰铋铫锍锒镂镏镙镥镧稆稂鸸鸾鹂鹩鹪疬疬痨瘌瘘瘰癯癞裢裣褸褴耒耢耧聆蛎蛉蜊蜡蠊蠡笠篥簏籁舻粞粼翎趔醑醪醴跺踉躏躐雱雏銮鎏鲈鲴鲢鲮鳟鳢髅魑鬣麟黧;m:妈麻玛码蚂马骂嘛吗埋买麦卖迈脉瞒馒蛮满蔓曼慢漫谩芒茫盲氓忙莽猫茅锚毛矛铆卯茂冒帽貌贸么玫枚梅酶霉煤没眉媒镁每美昧寐妹媚门闷们萌蒙檬盟锰猛梦孟眯醚靡糜迷谜弥米秘觅泌蜜密幂棉眠绵冕免勉娩缅面苗描瞄藐秒渺庙妙蔑灭民抿皿敏悯闽明螟鸣铭名命谬摸摹蘑模膜磨摩魔抹末莫墨默沫漠寞陌谋牟某拇牡亩姆母墓暮幕募慕木目睦牧穆乜半仫侔袭冂冥谟谧邙劢勐坶墁茉苜茆茛茭莓蓦薨瞢蘼扪咩咪哞唛喵暧幔茆岷嵋犸猕猸馍懵闵沐沔汨泖泯浇渑湎湨潸溟宀宓邈弭嫫嬷缈缙缦缪珉瑁枴杪楣殁昂暝牦耄毵脒腼朦旄焖熳祢愍懋懔淼碴礞眄眇眸瞥瞑钊钼镅镆镘秣鹋鹛瘼袂糖颟虻蛑蜢螨蟒蠓蠛簏艋蠓籼敉糸酪貊貘霾黾鏊鳗鳘鞔魅髦麽縻麋;n:拿哪呐钠那娜纳氖乃奶耐奈南男难囊挠脑恼闹淖呢馁内嫩能妮霓倪泥尼拟你匿腻逆溺蔫拈年碾撵捻念娘酿鸟尿捏聂孽啮镊镍涅您柠狞凝宁拧泞牛扭钮纽脓浓农弄奴努怒女暖虐疟挪懦糯诺廿孬孺佞侩佴侬傩讷隉坭垴埝芳莺荞蘖捺搦攘咛呶哝喏嗪嗯嚷囡狃猱镶怄怩甯弩妞胯驸琴弩瑙柰楠辇昵曩肭脲腩旎恣恁硪睨钕铌铙锘镎黏疒衲耨眝颞蛲蝻臬衄袅赧蹑鲇鲵鲶;o:哦欧鸥殴藕呕偶沤讴喔噢怄瓯耦;p:辟泊脯啪趴爬帕怕琶拍排牌徘湃派攀潘盘磐盼畔判叛乓庞旁耪胖抛咆刨炮袍跑泡呸胚培裴赔陪配佩沛喷盆砰抨烹澎彭蓬棚硼篷膨朋鹏捧碰坯砒霹批披劈琵毗啤脾疲皮匹痞僻屁譬篇偏片骗飘漂瓢票撇瞥拼频贫品聘乒坪苹萍平凭瓶评屏坡泼颇婆破魄迫粕剖扑铺仆莆葡菩蒲朴圃普浦谱曝瀑丕丿叵剽仳俜俳匍裒谝陂陴邳郸鄱圮埤堋鼙茎葩蒎匏邶怄衢犭犰悜悛愜愀慊憔阒阕阙丬戕沦淇溱骞搴褰逖遨道妾嫱骐绮绻缱缲琪琦璩杞棂榘楸樯槭檠樵檎辁戗耆赎觑挈氍肭豭欹炝袪祺恿愆憩硗碛磬磲畎钤铨镔锖锲锎镕鸽癯穹祫裎颀虔虮蚯蛩蜣蜻蜞蜡蝣蟾蠼罄筢笙箐篚箝衾裘羟糗綦綮觊跸跄蹊蹊謦瞿銮鲭鳅鳍鞒髂鬈麒黢黩鼢;r:然燃冉染瓤壤攘嚷让饶扰绕惹热壬仁人忍韧任认刃妊纫扔仍日戎茸蓉荣融熔溶容绒冗揉柔肉茹蠕儒孺如辱乳汝入褥软阮蕊瑞锐闰润若弱仜 仞偌芮冉菏茬甚蓐蓖蒿嚅嵘狨饪洳溽濡娆缛枘桡榕轫彤肮禳睿铞稔穰衽襦颥蚋蚺蜍箸糅蹂鞣髯;s:匙撒洒萨腮鳃塞赛三叁伞散桑嗓丧搔骚扫嫂瑟色涩森僧莎砂杀刹沙纱傻啥煞筛晒珊苫杉山删煽衫闪陕擅赡膳善汕扇缮墒伤商赏晌上尚裳梢捎稍烧芍勺韶少哨邵绍奢赊蛇舌舍赦摄射慑涉社设砷申呻伸身深娠绅神沈审婶甚肾慎渗声生甥牲升绳盛剩胜圣师失狮施湿诗尸虱十石拾时什食蚀实识史矢使屎驶始式示士世柿事拭誓逝势是嗜噬适仕侍释饰氏市恃室视试收手首守寿授售受瘦兽蔬枢梳殊抒输叔舒淑疏书赎孰熟薯暑曙署蜀黍鼠属术述树束戍竖墅庶数漱恕刷耍摔衰甩帅栓拴霜双爽谁水睡税吮瞬顺舜说硕朔烁斯撕嘶思私司丝死肆寺嗣四似饲巳松耸怂颂送宋讼诵搜艘擞嗽苏酥俗素速粟僳塑溯诉肃酸蒜算虽隋随绥髓碎岁穗遂隧祟孙损笋蓑梭唆缩琐索锁所厦卅啻厍厮剜仨倏佘夙兕凇讹诌谗谧谇谖鄯劭叟厶塾涎垌堋埽芟苕荪莳荽莘菘菽蓄蒴藐薮扌摅搠搡弒唑哂唢嗖嗽嗫嗄嗍嗾瘛崧嵫嵩彡狩狲狻猞飧饣馒馓忪悚憟闩氵汜沭泗涞淞渖涮溲滠澍澌潜潲濉邃妁姒姗姝娑嬗孀骋骟纟纾绡绶缌缫秒椹榫槊殇轼贰挲氅毓戚胂脐腧臊膻歃歙飒飨殳炻熵燧衤祀磉眚眭睃睢瞍畲钐铄铈铎铯锶锪锢矧秫穑鸯疝痱瘙竦耜颡蛎屦蛸螋螯蟀蟮舐笙笥篮筲簌舢艄艏裟羧糁酾豕跚觞觫霎隼稣鲋鲨鲴鳋鳝麝;t:塌他它她塔獭挞蹋踏胎苔抬台泰酞太态汰坍摊贪瘫滩坛檀痰潭谭谈坦毯袒碳探叹炭汤塘搪堂棠膛唐糖倘躺淌趟烫掏涛滔绦萄桃逃淘陶讨套特藤腾疼誊梯剔踢锑提题蹄啼体替嚏惕涕剃屉天添填田甜恬舔腆挑条迢眺跳贴铁帖厅听烃汀廷停亭庭挺艇通桐酮瞳同铜彤童桶捅筒统痛偷投头透凸秃突图徒途涂屠土吐兔湍团推颓腿蜕褪退吞

屯臀拖托脱鸵陀驮驼椭妥拓唾毛鬟佟佗佻倜傥僮仝汆亠郜郯坨堍黄茼莛荼菇菟葶薹抟拯忒啕嗵帑饨庹恸悌忝闼阗洵沱洮溻溏潼逖遢彖婷绨缇瑭韬柝柁梃榻樘橐殄昙曒肽豚滕贠熄焘祧恙忐慝砼砣町畋疃钍钛钽铊铰钖锍锬镗鹆窕裼鹣覃蜓蜩螗螳笤篫崁崧酡酴醌醣趿跎跆霆龆鼋鲍鲧鲳骰餮饕髫;w:挖哇蛙洼娃瓦袜歪外豌弯湾玩顽丸烷完碗挽晚皖惋宛婉万腕汪王亡枉网往旺望忘妄威巍微危韦违桅围唯惟潍维苇萎委伟伪尾纬未蔚味畏胃喂魏位渭谓尉慰卫瘟温蚊文闻纹吻稳紊问嗡翁瓮挝蜗涡窝我斡卧握沃巫呜钨乌污诬屋无芜梧吾吴毋武五捂午舞伍侮坞戊雾晤物勿务悟误兀刎剐罔佤仵倭偎诿阢隈隗邬圩圬芄芟莴菟葳蓊薇薷尢唔冂帏帷幄崴嵬猥猬庑怃忤惘闱阌汶沩洧浯涠渥寤迮逶妩娲娓婺骜纨绾玮玟琬璎鼾杌辋軎牾肪脘腽炜焐煨硪畹鹇鹜痦瘘蜈蜿艉雟龌鉴鲔魍鼯;x:昔熙析西硒矽晰嘻吸锡牺稀息希悉膝夕惜熄烯溪汐犀檄袭席习媳喜铣洗系隙戏细瞎虾匣霞辖暇峡侠狭下夏吓掀锨先仙鲜纤咸贤衔舷闲涎弦嫌显险现献县腺馅羡宪陷限线厢镶香箱襄湘乡翔祥详想响享项巷橡像向象萧硝霄削哮嚣销消宵淆晓小孝校肖啸笑效楔些歇蝎鞋协挟携邪斜胁谐写械卸蟹懈泄泻谢屑薪芯锌欣辛新忻心信衅星腥猩惺兴刑型形邢醒幸杏性姓兄凶胸匈汹雄熊休修羞朽嗅锈秀袖绣墟戌需虚嘘须徐许蓄酗叙旭序恤絮婿绪续轩喧宣悬旋玄选癣眩绚靴薛学穴雪血勋熏循旬询寻驯巡殉汛训讯逊迅吁凶偕僖儇兮巽亵冼诩谑谖陉隰郗勖勰燮埙馨芎芑芟苋荇荀荥荨莶莳葙葸萱蓿蓴菪薰蘚奚揎擷擄呷哓咻嗥岘岫峋嵴徜徙狎猃獬犷饧饷馓庥庠廨忡 恟悻阋泫洫洵浔浠淅潠溆渲溴潇漩瀣逍遐暹邂屣娴嬉骁骧绌绡缃缬顼玺瑕璇枭柙栲栩楦榭榍樨昕暄曛曦砚氙欹歆炫煊煞煦熹祆禊禧荥硎硖碹盱铉镟晢鹇鹆痃痫窈胥蚬蜥螅蟋蟓罅筢筱箫舄艋羲籼粞糈翕酰醑醢醺踅跌跹躐貊霰鑫鲞鲟鳃魈飧鬃黠鼷;y:压押鸦鸭呀丫芽牙蚜崖衙涯雅哑亚讶焉咽阉烟淹盐严研蜒岩延言颜阎炎沿奄掩眼衍演艳堰燕厌砚雁唁彦焰宴谚验殃央鸯秧杨扬佯疡羊洋阳氧仰痒养样漾邀腰妖瑶摇尧遥窑谣姚咬舀药耀椰噎耶爷野冶也页掖业叶曳腋夜液一壹医揖铱依伊衣颐夷遗移仪胰疑沂宜姨彝椅蚁倚已乙矣以艺抑易邑屹亿役臆逸肄疫亦裔意毅忆义益溢诣议谊译异翼翌绎茵荫因殷音阴姻吟银淫寅饮尹引隐印英樱婴鹰缨莹萤营荧蝇迎赢盈影颖硬映哟拥佣臃痈庸雍踊蛹咏泳涌永恿勇用幽优悠忧尤由邮铀犹油游酉有友右佑釉诱又幼迂淤于盂榆虞愚舆余俞逾鱼愉渝渔隅予娱雨屿禹宇语羽玉域芋郁遇喻峪御愈欲狱育誉浴寓裕预豫驭鸳渊冤元垣袁原援辕园员圆猿源缘远苑愿怨院曰约越跃钥岳粤月悦阅耘云郧匀陨允运蕴酝晕韵孕轧禺夭爻胤毓厍醑卣刈劓伋伢攸佚侑俏俨俣俑偃龠兖赢讠诒诙谒谕谖邶郓郢郾鄢廴壅圯垭垠垴埸堙塬嚏芫芸苡茆茔莠莜筱蓥蓠蕨弈奕挹揶揠揄掾撄弋吆呓吪哄咦咿唷喁喑嘤噫噎囿圄圉岈峄崦崾嵛嶷徉徭犹狻猃猗夤饩饴馀庾膺怏怿怡恢恽悒愠慵闫阈阕沅泱涸湮滟滠漪瀛瀹宥迂迤鬻妍妪妤娅媛嫣驿纡纭缢幺邕琊珧瑛琰瑜瑗璎韫杳柚桠楹榱橡檐猷殒殪轶轫昀昱晔晏曜贻觇氩氤氲牖爰刖肴胭腌腴媵欻於旖炀烨烊焱煜熨熘燠恙尹聿研督眙睚罨钇钰铖铕铘铞铪镛镱甬鸢鹆鹞鹬鹬疣痂瘐瘐瘀瘁瘿瘾癔翊窈窬窖窒寇衤颍蚓蚰蚴蛘蜮蜴蝓蝣螈罂竽筵筠筧舆舁舣羿翳舔酏酽雩霪龈龋鼋鲂鳐鳙鞅魇餍镬黝黟鼬鼾;z:匝砸杂栽哉灾宰载再在咱攒暂赞赃脏葬遭糟凿藻枣早澡蚤躁噪造皂灶燥责择则泽贼怎增憎赠扎喳渣札铡闸眨栅榨咋乍炸诈摘斋宅窄债寨瞻毡詹粘沾盏斩辗崭展蘸栈占战站湛绽樟章彰漳张掌涨杖丈帐账仗胀瘴障招昭找沼赵照罩兆肇召遮折哲蛰辙者锗蔗这浙珍斟真甄砧臻贞针侦枕疹诊震振镇阵蒸挣睁征狰争怔整拯正政帧症郑证芝枝支吱蜘知肢脂汁之织职直植殖执值侄址指止趾只旨纸志挚掷至致置帜峙制智秩稚质炙痔滞治窒中盅忠钟衷终种肿仲众舟周州洲诌粥轴肘帚咒皱宙昼骤珠株蛛朱猪诸诛逐竹烛煮拄瞩嘱主著柱助蛀贮铸筑住注祝驻抓爪拽专砖转撰赚篆桩庄装妆撞壮状椎锥追赘坠缀谆准捉拙卓桌琢茁酌啄着灼浊兹咨资姿滋淄孜紫仔籽滓子自渍字鬃棕踪宗综总纵邹走奏揍租足卒族祖诅阻组钻纂嘴醉最罪尊遵昨左佐柞做作坐座卮丶仄赜仉伫侏倬傯俎冢诏诤诹诼谘谪谮谗阼陟陬郅郑鄚鄹圳堉芷芒茱药菹蓁蕞奘拶揸摭摺撙擢攥吒咂哳咤啐啧啭哨嘬帙帻幛峥崽嵫嶂徵獐夂馔忮怍惴浈洙浞渚涿潴濯迮彟咫姊妯嫜孳驻驺骓鹭纡绉缁缒缜缯缵笛璋瓒杼柞栀枧栀桎桢梓棹楱楂榛槠檫樽轼轸轻辄辎臧甑炭咎贽赁贩肫胄胨胗胝朕踪膛旃灶祉祚祇祯禚恣斫砟砦碡磔粞凿鲔毗眵瞢钊钲铢铮锃锴镝镯锺雉秫稹鸠鹗鹪痄疰痣瘃瘵瘭窀褶耔颛蚱蛭蜇螽蟑竺笊笫笮筝篑箸箴簪籀舯舳舭溆粽糌翥絮趔趱赭酎酢酯跎踬踯踵躅躜豸觜觯訾龇鲽鳟髭麈鼯';
const EXTRA: string = '藏=cz;长=cz;朝=cz;传=cz;单=ds;还=hh;降=jx;解=jx;乐=ly;奇=qj;区=qo;省=sx;宿=sx;为=ww;相=xx;行=hx;要=yy;应=yy;与=yy;曾=cz;重=cz';
let _map: Map<string, string> | null = null;
/** 构建 字→声母 映射（多音字为多个声母连写，如「长」=「cz」） */
function buildMap(): Map<string, string> {

```
// ……（此处省略中间 373 页源码，按登记要求提交前 30 页与后 30 页）……
    [22, 3, 36, 2, 37],
    [26, 4, 43, 1, 44]
  ];
  /** 对齐图案中心坐标 */
  private static readonly ALIGN: number[][] = [
    [], [6, 18], [6, 22], [6, 26], [6, 30],
    [6, 34], [6, 22, 38], [6, 24, 42], [6, 26, 46], [6, 28, 50]
  ];
  /** 版本升序，返回 true=黑 的模块矩阵；放不下返回空数组 */
  static make(text: string): boolean[][] {
    const bytes: number[] = QrCode.utf8(text);
    const v: number = QrCode.pickVersion(bytes.length);
    if (v < 1) {
      return [];
    }
    let bestMask: number = 0;
    let bestScore: number = -1;
    for (let mask = 0; mask < 8; mask++) {
      const m: number[][] = QrCode.build(bytes, v, mask);
      const s: number = QrCode.penalty(m);
      if (bestScore < 0 || s < bestScore) {
        bestScore = s;
        bestMask = mask;
      }
    }
    const grid: number[][] = QrCode.build(bytes, v, bestMask);
    const out: boolean[][] = [];
    for (let r = 0; r < grid.length; r++) {
      const row: boolean[] = [];
      for (let c = 0; c < grid[r].length; c++) {
        row.push(grid[r][c] === 1);
      }
      out.push(row);
    }
    return out;
  }
  private static pickVersion(n: number): number {
    for (let v = 1; v <= QrCode.EC.length; v++) {
      const cap: number = QrCode.dataCapacity(v);
      const header: number = v <= 9 ? 12 : 20;   // 模式 4bit + 计数 8/16bit
      if (n * 8 + header <= cap * 8) {
        return v;
      }
    }
    return -1;
  }
  private static dataCapacity(v: number): number {
    const p: number[] = QrCode.EC[v - 1];
    return p[1] * p[2] + p[3] * p[4];
```

```
}
private static utf8(text: string): number[] {
  const enc: util.TextEncoder = new util.TextEncoder();
  const buf: Uint8Array = enc.encodeInto(text);
  const out: number[] = [];
  for (let i = 0; i < buf.length; i++) {
    out.push(buf[i]);
  }
  return out;
}
// ==================== 码字 ====================
private static encodeData(bytes: number[], v: number): number[] {
  const cap: number = QrCode.dataCapacity(v);
  const bits: number[] = [];
  const push = (val: number, len: number): void => {
    for (let i = len - 1; i >= 0; i--) {
      bits.push((val >>> i) & 1);
    }
  };
  push(4, 4);                                    // 字节模式 0100
  push(bytes.length, v <= 9 ? 8 : 16);
  for (let i = 0; i < bytes.length; i++) {
    push(bytes[i], 8);
  }
  const total: number = cap * 8;
  for (let i = 0; i < 4 && bits.length < total; i++) {
    bits.push(0);                                // 终止符
  }
  while (bits.length % 8 !== 0) {
    bits.push(0);                                // 补齐到字节边界
  }
  const cw: number[] = [];
  for (let i = 0; i < bits.length; i += 8) {
    let b: number = 0;
    for (let j = 0; j < 8; j++) {
      b = (b << 1) | bits[i + j];
    }
    cw.push(b);
  }
  let k: number = 0;
  while (cw.length < cap) {
    cw.push(k % 2 === 0 ? 0xEC : 0x11);          // 填充码字
    k++;
  }
  return cw;
}
private static buildCodewords(bytes: number[], v: number): number[] {
  const data: number[] = QrCode.encodeData(bytes, v);
  const p: number[] = QrCode.EC[v - 1];
  const ec: number = p[0];
```

```
  const g1n: number = p[1];
  const g1d: number = p[2];
  const g2n: number = p[3];
  const g2d: number = p[4];
  const blocks: number[][] = [];
  let pos: number = 0;
  for (let i = 0; i < g1n; i++) {
    blocks.push(data.slice(pos, pos + g1d));
    pos += g1d;
  }
  for (let i = 0; i < g2n; i++) {
    blocks.push(data.slice(pos, pos + g2d));
    pos += g2d;
  }
  const eccs: number[][] = [];
  for (let i = 0; i < blocks.length; i++) {
    eccs.push(QrCode.rsEncode(blocks[i], ec));
  }
  const out: number[] = [];
  const maxData: number = g1d > g2d ? g1d : g2d;
  for (let i = 0; i < maxData; i++) {
    for (let b = 0; b < blocks.length; b++) {
      if (i < blocks[b].length) {
        out.push(blocks[b][i]);
      }
    }
  }
  for (let i = 0; i < ec; i++) {
    for (let b = 0; b < eccs.length; b++) {
      out.push(eccs[b][i]);
    }
  }
  return out;
}
// ---- GF(256) / Reed-Solomon ----
private static gfMul(x: number, y: number): number {
  let z: number = 0;
  for (let i = 7; i >= 0; i--) {
    z = (z << 1) ^ ((z >>> 7) * 0x11D);
    z ^= ((y >>> i) & 1) * x;
  }
  return z & 0xFF;
}
private static gexp(n: number): number {
  let x: number = 1;
  for (let i = 0; i < n; i++) {
    x = QrCode.gfMul(x, 2);
  }
  return x;
}
```

```
private static rsEncode(data: number[], ecLen: number): number[] {
  // 生成多项式 ∏(x - α^i)
  let poly: number[] = [1];
  for (let i = 0; i < ecLen; i++) {
    const next: number[] = [];
    for (let j = 0; j <= poly.length; j++) {
      next.push(0);
    }
    for (let j = 0; j < poly.length; j++) {
      next[j] ^= poly[j];
      next[j + 1] ^= QrCode.gfMul(poly[j], QrCode.gexp(i));
    }
    poly = next;
  }
  const res: number[] = [];
  for (let i = 0; i < data.length + ecLen; i++) {
    res.push(i < data.length ? data[i] : 0);
  }
  for (let i = 0; i < data.length; i++) {
    const factor: number = res[i];
    if (factor === 0) {
      continue;
    }
    for (let j = 0; j < poly.length; j++) {
      res[i + j] ^= QrCode.gfMul(poly[j], factor);
    }
  }
  const out: number[] = [];
  for (let i = data.length; i < res.length; i++) {
    out.push(res[i]);
  }
  return out;
}
// ==================== 矩阵 ====================
private static build(bytes: number[], v: number, mask: number): number[][] {
  const n: number = 17 + 4 * v;
  const m: number[][] = [];
  const fn: boolean[][] = [];
  for (let r = 0; r < n; r++) {
    const row: number[] = [];
    const frow: boolean[] = [];
    for (let c = 0; c < n; c++) {
      row.push(0);
      frow.push(false);
    }
    m.push(row);
    fn.push(frow);
  }
  const set = (r: number, c: number, val: number): void => {
    if (r >= 0 && r < n && c >= 0 && c < n) {
```

```
        m[r][c] = val;
        fn[r][c] = true;
      }
    };
    // 三个定位图案（含分隔带）
    QrCode.finder(set, 0, 0);
    QrCode.finder(set, 0, n - 7);
    QrCode.finder(set, n - 7, 0);
    // 时序图案
    for (let i = 8; i < n - 8; i++) {
      const bit: number = i % 2 === 0 ? 1 : 0;
      set(6, i, bit);
      set(i, 6, bit);
    }
    // 校正图案
    const pos: number[] = QrCode.ALIGN[v - 1];
    for (let a = 0; a < pos.length; a++) {
      for (let b = 0; b < pos.length; b++) {
        const r: number = pos[a];
        const c: number = pos[b];
        if ((r === 6 && c === 6) || (r === 6 && c === n - 7) || (r === n - 7 && c
=== 6)) {
          continue;
        }
        for (let dr = -2; dr <= 2; dr++) {
          for (let dc = -2; dc <= 2; dc++) {
            const far: number = Math.max(Math.abs(dr), Math.abs(dc));
            set(r + dr, c + dc, far !== 1 ? 1 : 0);
          }
        }
      }
    }
    // 预留格式信息区与固定黑点
    for (let i = 0; i < 9; i++) {
      if (!fn[8][i]) {
        set(8, i, 0);
      }
      if (!fn[i][8]) {
        set(i, 8, 0);
      }
    }
    for (let i = 0; i < 8; i++) {
      if (!fn[8][n - 1 - i]) {
        set(8, n - 1 - i, 0);
      }
      if (!fn[n - 1 - i][8]) {
        set(n - 1 - i, 8, 0);
      }
    }
    set(n - 8, 8, 1);
```

```
// 版本信息（v>=7）
if (v >= 7) {
  const bits: number = QrCode.bchTypeNumber(v);
  for (let i = 0; i < 18; i++) {
    const bit: number = (bits >>> i) & 1;
    set(Math.floor(i / 3), i % 3 + n - 11, bit);
    set(i % 3 + n - 11, Math.floor(i / 3), bit);
  }
}
// 数据按之字形填充
const cw: number[] = QrCode.buildCodewords(bytes, v);
let inc: number = -1;
let row: number = n - 1;
let bitIndex: number = 7;
let byteIndex: number = 0;
let col: number = n - 1;
while (col > 0) {
  if (col === 6) {
    col--;
  }
  while (true) {
    for (let k = 0; k < 2; k++) {
      const cc: number = col - k;
      if (m[row][cc] === 0 && !fn[row][cc]) {
        let dark: number = 0;
        if (byteIndex < cw.length) {
          dark = (cw[byteIndex] >>> bitIndex) & 1;
        }
        if (QrCode.maskOn(mask, row, cc)) {
          dark = dark ^ 1;
        }
        m[row][cc] = dark;
        fn[row][cc] = true;
        bitIndex--;
        if (bitIndex === -1) {
          byteIndex++;
          bitIndex = 7;
        }
      }
    }
    row += inc;
    if (row < 0 || row >= n) {
      row -= inc;
      inc = -inc;
      break;
    }
  }
  col -= 2;
}
// 格式信息（纠错等级 M = 00）
```

```
    const fmt: number = QrCode.bchTypeInfo((0 << 3) | mask);
    for (let i = 0; i < 15; i++) {
      const bit: number = (fmt >>> i) & 1;
      if (i < 6) {
        m[i][8] = bit;
      } else if (i < 8) {
        m[i + 1][8] = bit;
      } else {
        m[n - 15 + i][8] = bit;
      }
    }
    for (let i = 0; i < 15; i++) {
      const bit: number = (fmt >>> i) & 1;
      if (i < 8) {
        m[8][n - i - 1] = bit;
      } else if (i < 9) {
        m[8][15 - i] = bit;
      } else {
        m[8][15 - i - 1] = bit;
      }
    }
    m[n - 8][8] = 1;
    return m;
  }
  private static finder(set: (r: number, c: number, v: number) => void, r0: number
, c0: number): void {
    for (let r = -1; r <= 7; r++) {
      for (let c = -1; c <= 7; c++) {
        const edge: boolean = (r >= 0 && r <= 6 && (c === 0 || c === 6)) ||
          (c >= 0 && c <= 6 && (r === 0 || r === 6));
        const core: boolean = r >= 2 && r <= 4 && c >= 2 && c <= 4;
        set(r0 + r, c0 + c, edge || core ? 1 : 0);
      }
    }
  }
  private static maskOn(mask: number, i: number, j: number): boolean {
    if (mask === 0) {
      return (i + j) % 2 === 0;
    }
    if (mask === 1) {
      return i % 2 === 0;
    }
    if (mask === 2) {
      return j % 3 === 0;
    }
    if (mask === 3) {
      return (i + j) % 3 === 0;
    }
    if (mask === 4) {
      return (Math.floor(i / 2) + Math.floor(j / 3)) % 2 === 0;
```

```
  }
  if (mask === 5) {
    return ((i * j) % 2) + ((i * j) % 3) === 0;
  }
  if (mask === 6) {
    return (((i * j) % 2) + ((i * j) % 3)) % 2 === 0;
  }
  return (((i * j) % 3) + ((i + j) % 2)) % 2 === 0;
}
private static bchDigit(data: number): number {
  let d: number = 0;
  let v: number = data;
  while (v !== 0) {
    d++;
    v = v >>> 1;
  }
  return d;
}
private static bchTypeInfo(data: number): number {
  const G15: number = 0x537;
  const G15_MASK: number = 0x5412;
  let d: number = data << 10;
  while (QrCode.bchDigit(d) - QrCode.bchDigit(G15) >= 0) {
    d ^= G15 << (QrCode.bchDigit(d) - QrCode.bchDigit(G15));
  }
  return ((data << 10) | d) ^ G15_MASK;
}
private static bchTypeNumber(data: number): number {
  const G18: number = 0x1F25;
  let d: number = data << 12;
  while (QrCode.bchDigit(d) - QrCode.bchDigit(G18) >= 0) {
    d ^= G18 << (QrCode.bchDigit(d) - QrCode.bchDigit(G18));
  }
  return (data << 12) | d;
}
/** 掩码评分：四条标准规则，分数越低越好扫 */
private static penalty(m: number[][]): number {
  const n: number = m.length;
  let score: number = 0;
  // 规则1：连续同色 5 个以上
  for (let i = 0; i < n; i++) {
    let run: number = 1;
    for (let j = 1; j < n; j++) {
      if (m[i][j] === m[i][j - 1]) {
        run++;
      } else {
        if (run >= 5) {
          score += 3 + (run - 5);
        }
        run = 1;
```

```
      }
    }
    if (run >= 5) {
      score += 3 + (run - 5);
    }
    run = 1;
    for (let j = 1; j < n; j++) {
      if (m[j][i] === m[j - 1][i]) {
        run++;
      } else {
        if (run >= 5) {
          score += 3 + (run - 5);
        }
        run = 1;
      }
    }
    if (run >= 5) {
      score += 3 + (run - 5);
    }
  }
  // 规则2：2x2 同色块
  for (let r = 0; r < n - 1; r++) {
    for (let c = 0; c < n - 1; c++) {
      const v0: number = m[r][c];
      if (v0 === m[r][c + 1] && v0 === m[r + 1][c] && v0 === m[r + 1][c + 1]) {
        score += 3;
      }
    }
  }
  // 规则3：1:1:3:1:1 图形（定位图案的样子）
  const pat: number[] = [1, 0, 1, 1, 1, 0, 1];
  for (let r = 0; r < n; r++) {
    score += QrCode.patScore((i: number): number => m[r][i], n, pat);
  }
  for (let c = 0; c < n; c++) {
    score += QrCode.patScore((i: number): number => m[i][c], n, pat);
  }
  // 规则4：黑白比例偏离 50%
  let dark: number = 0;
  for (let r = 0; r < n; r++) {
    for (let c = 0; c < n; c++) {
      if (m[r][c] === 1) {
        dark++;
      }
    }
  }
  const ratio: number = Math.abs(dark * 100 / (n * n) - 50);
  score += Math.floor(ratio / 5) * 10;
  return score;
}
```

```
  private static patScore(get: (i: number) => number, n: number, pat: number[]): n
umber {
    let score: number = 0;
    for (let i = 0; i + 7 <= n; i++) {
      let hit: boolean = true;
      for (let k = 0; k < 7; k++) {
        if (get(i + k) !== pat[k]) {
          hit = false;
          break;
        }
      }
      if (!hit) {
        continue;
      }
      let before: boolean = true;
      let after: boolean = true;
      for (let k = i - 4; k < i; k++) {
        if (k >= 0 && get(k) !== 0) {
          before = false;
        }
      }
      for (let k = i + 7; k < i + 11; k++) {
        if (k < n && get(k) !== 0) {
          after = false;
        }
      }
      if (before || after) {
        score += 40;
      }
    }
    return score;
  }
}
// ================================================================================
// 文件：entry/src/main/ets/common/Tip.ets     （共 96 行）
// ================================================================================
/**
 * 全局轻提示（OSD）。
 *
 * 为什么不用系统的 promptAction.showToast：
 *  1. TV 是横屏全屏 + 隐藏系统栏，系统 toast 固定贴底，在 3 米观看距离下字号太小
;
 *  2. 样式跟应用整体（深色 + 白描边）不一致，看着像外来弹窗；
 *  3. 部分大屏固件会把 toast 画在解码层下面，直接看不见。
 * 所以自绘一层，位置固定在下三分之一处、字号 20、胶囊底 + 白描边，远看清晰。
 *
 * 用法（任何页面）：
 *   @StorageLink(TIP_MSG) tipMsg: string = '';
 *   build() 最外层 Stack 里加： TipLayer({ msg: this.tipMsg })
 *   触发：Tip.show('已收藏')
```

```
 *
 * 注意：TipLayer 必须是 @Component + @Prop，不能写成 @Builder 函数。
 * @Builder 的参数是**按值传递**，状态变了不会重建 —— 表现就是「气泡壳子出来了、
 * 文字却是空的」或「连着两条提示时文字停在上一条」。直播页用 visibility 控制显隐
 * （不是 if 重建），首帧 tipMsg 是空串，于是气泡可见但一个字都没有。
 *
 * 只用一个 AppStorage 键，所以同一时刻全应用只会有一条提示，
 * 后一条直接覆盖前一条 —— 这正是遥控器快速连按时期望的行为。
 */
import { T } from './Model';
import { Device } from './Device';
/** AppStorage 键名，页面用 @StorageLink 绑定 */
export const TIP_MSG: string = 'tipMsg';
/** 提示停留时长（ms）。比系统默认 1500 长一点，大屏上给人读完的时间 */
const TIP_HOLD: number = 1800;
export class Tip {
  private static timer: number = -1;
  /** 弹一条提示。同一条文字重复触发不会闪（覆盖后重新计时） */
  static show(msg: string): void {
    if (msg.length === 0) {
      return;
    }
    AppStorage.setOrCreate<string>(TIP_MSG, msg);
    if (Tip.timer >= 0) {
      clearTimeout(Tip.timer);
      Tip.timer = -1;
    }
    Tip.timer = setTimeout(() => {
      AppStorage.setOrCreate<string>(TIP_MSG, '');
      Tip.timer = -1;
    }, TIP_HOLD);
  }
  /** 立即收起，切页时用，避免提示串到下一个页面 */
  static hide(): void {
    if (Tip.timer >= 0) {
      clearTimeout(Tip.timer);
      Tip.timer = -1;
    }
    AppStorage.setOrCreate<string>(TIP_MSG, '');
  }
}
/**
 * 提示层。放在页面最外层 Stack 的最后，盖在内容之上、不挡点击。
 * hitTestBehavior 用 Transparent：能看见但不吃事件，底下的按钮照常可点。
 *
 * 用 @Component + @Prop 而不是 @Builder：@Prop 会跟着页面上的 tipMsg 一起刷新，
 * 从「空 → 有字」「A → B」都能正确更新（@Builder 的按值参数两者都做不到）。
 */
@Component
export struct TipLayer {
```

```
  @Prop msg: string = '';
  build() {
    Column() {
      Row() {
        Text(this.msg)
          .fontSize(20)
          .fontColor(T.TXT)
          .maxLines(2)
          .textOverflow({ overflow: TextOverflow.Ellipsis })
          .textAlign(TextAlign.Center)
      }
      .constraintSize({ maxWidth: Device.isPhone() ? '90%' : 720 })
      .padding({ left: 28, right: 28, top: 14, bottom: 14 })
      .borderRadius(T.RADIUS_PILL)
      .backgroundColor(T.BLACK_80)
      .border({ width: T.BORDER, color: T.WHITE_30 })
    }
    .width('100%')
    .height('100%')
    .padding({ bottom: 120 })
    .justifyContent(FlexAlign.End)
    .alignItems(HorizontalAlign.Center)
    .hitTestBehavior(HitTestMode.Transparent)
  }
}
// ================================================================================
// 文件：entry/src/main/ets/common/TvFocus.ets     （共 23 行）
// ================================================================================
/**
 * TV 遥控器焦点适配 —— 公共基础设施。
 *
 * ArkUI 的遥控器焦点「三件套」（接线在各页面，这里只放常量与辅助）：
 *   1. 可交互元素加 .focusable(true)：方向键能聚焦它，聚焦后按 OK/确定 键自动触发
它的 onClick。
 *   2. 页面首个可交互元素加 .defaultFocus(true)：进入页面时焦点自动落在这里。
 *   3. 焦点视觉反馈：onFocus/onBlur 驱动与 onHover 相同的高亮状态（鼠标悬停与遥控
器焦点并存）。
 *
 * 方向键在可聚焦组件间的移动、焦点跟随滚动到可见，均由 ArkUI 自动完成，无需手写导
航。
 * 只有「返回」「菜单」这类全局语义键需要 onKeyEvent 显式处理（用下面的辅助函数判
断）。
 */
import { KeyCode } from '@ohos.multimodalInput.keyCode';
/** 是否为「返回」键（遥控器 BACK，键值 2）。页面级 onKeyEvent 里据此决定 pop 或退
出 */
export function isBackCode(code: number): boolean {
  return code === KeyCode.KEYCODE_BACK;
}
/** 是否为「菜单」键（部分遥控器右上角菜单键） */
```

```
export function isMenuCode(code: number): boolean {
  return code === KeyCode.KEYCODE_MENU;
}
// ============================================================================
// 文件：entry/src/main/ets/entryability/EntryAbility.ets     （共 80 行）
// ============================================================================
import UIAbility from '@ohos.app.ability.UIAbility';
import AbilityConstant from '@ohos.app.ability.AbilityConstant';
import Want from '@ohos.app.ability.Want';
import window from '@ohos.window';
import { BusinessError } from '@ohos.base';
import { Store } from '../common/Store';
import { Device } from '../common/Device';
import { Feedback } from '../common/Feedback';
import { PlayGuard } from '../common/PlayGuard';
import { Net } from '../common/Net';
import { Theme } from '../common/Theme';
export default class EntryAbility extends UIAbility {
  onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    // 必须先于 loadContent 判定设备形态，否则首页第一帧会按错误的布局渲染
    Device.init();
    Store.init(this.context);
    // 主题必须在 loadContent 之前定好：首帧是按 `T` 上的色板画的，
    // 放到页面的 aboutToAppear 里就晚了（异步返回时首帧已经画完）
    Theme.loadSync(this.context);
    // 加载点击反馈的音效（rawfile click.wav）
    Feedback.init(this.context);
    // 网络状态：断网/恢复各提示一次，避免多个页面同时报「网络错误」
    Net.init();
  }
  onWindowStageCreate(windowStage: window.WindowStage): void {
    windowStage.loadContent('pages/Home', (err: BusinessError) => {
      if (err.code) {
        return;
      }
      try {
        const win = windowStage.getMainWindowSync();
        win.setWindowKeepScreenOn(true);
        // 与 FongMi 的 splash_background 同色，避免启动瞬间白闪
        win.setWindowBackgroundColor('#141218');
        if (Device.isTv()) {
          // 电视：沉浸式全屏 + 隐藏系统栏 + 强制横屏
          win.setWindowLayoutFullScreen(true);
          win.setWindowSystemBarEnable([]);
          try {
            win.setPreferredOrientation(window.Orientation.LANDSCAPE);
          } catch (e) {
            // 个别系统不支持运行时锁方向，忽略
          }
        } else {
```

```
          // 手机：内容不沉浸到状态栏后面（避开刘海/状态栏占用区），
          // 只保留深色状态栏、隐藏导航栏（底部已有应用自己的 Tab）。
          // 否则 setWindowLayoutFullScreen(true) 会把顶部标题栏顶到屏幕最上沿，贴
住刘海。
          win.setWindowLayoutFullScreen(false);
          win.setWindowSystemBarEnable(['status']);
          try {
            win.setWindowSystemBarProperties({
              statusBarColor: '#141218',
              statusBarContentColor: '#FFFFFF',
              navigationBarColor: '#1C1B1F'
            });
          } catch (e) {
            // 个别系统不支持状态栏配色，忽略
          }
        }
      } catch (e) {
        // 部分电视系统不支持这些窗口能力，忽略
      }
    });
  }
  /**
   * 应用退到后台 / 回桌面。电视上「退出后还在出声」的兜底就在这里：
   * 页面级的 onPageHide 在部分 TV 系统上触发不可靠，而 Ability 切后台一定会走这里
。
   */
  onBackground(): void {
    PlayGuard.stopAll();
  }
  onDestroy(): void {
    PlayGuard.stopAll();
  }
}
// ============================================================================
// 文件：entry/src/main/ets/pages/WebdavPage.ets     （共 801 行）
// ============================================================================
/**
 * WebDAV 网盘 —— 挂载 WebDAV 服务器，浏览目录并识别影视资源。
 *
 * 三种视图：
 *   sources —— 源列表（添加 / 编辑 / 删除 / 进入）
 *   browse  —— 浏览目录（点目录进入、点视频播放），可一键「识别影视」
 *   scan    —— 识别结果（按片名聚合的电影/剧集），点开选集播放
 *
 * 播放走 Player 的「直连」链路，通过 PlayParam.header 把 Basic Auth 带过去，
 * 剧集用 Playlist 共享，遥控器可切上/下一集。
 */
import { Feedback } from '../common/Feedback';
import router from '@ohos.router';
import common from '@ohos.app.ability.common';
```

```
import { Store } from '../common/Store';
import { PlayParam, WebdavSource, WebdavEntry, WebdavVod, WebdavEp, Episode, WdHis
tory, T } from '../common/Model';
import { Webdav } from '../common/Webdav';
import { Playlist } from '../common/Playlist';
import { Device, KEY_IS_PHONE } from '../common/Device';
import { Theme, KEY_THEME } from '../common/Theme';
import { WdQrPanel } from '../common/WdQr';
import { Tip, TIP_MSG, TipLayer } from '../common/Tip';
@Entry
@Component
struct WebdavPage {
  @State sources: WebdavSource[] = [];
  @State view: string = 'sources';
  @State hoverKey: string = '';
  @State loading: boolean = false;
  @State msg: string = '';
  @State msgColor: string = T.SUB;
  // 当前浏览的源与路径
  @State cur: WebdavSource = new WebdavSource();
  @State curPath: string = '/';
  @State entries: WebdavEntry[] = [];
  // 识别结果
  @State vods: WebdavVod[] = [];
  // 网盘续播记录
  @State wdHist: WdHistory[] = [];
  // 选集弹窗
  @State epOpen: boolean = false;
  @State selVod: WebdavVod = new WebdavVod();
  @StorageProp(KEY_IS_PHONE) isPhone: boolean = false;
  /** 当前主题。build 里必须用到（见 .id），否则换了色板不会重绘 */
  @StorageProp(KEY_THEME) themeMode: string = 'dark';
  @StorageLink(TIP_MSG) tipMsg: string = '';
  /** 电视端扫码添加网盘面板 */
  @State wdQrOpen: boolean = false;
  async aboutToAppear(): Promise<void> {
    await Store.init(getContext(this) as common.UIAbilityContext);
    await this.refreshData();
  }
  /** 从播放页/设置页返回时刷新源列表与续播记录 */
  onPageShow(): void {
    this.refreshData();
    // 播放页退出时续播记录是异步落盘的（saveWdPos fire-and-forget），
    // 刚回来那一刻 prefs 可能还没 flush 完，读到旧值 → 「继续观看」不显示。
    // 延迟再读一次，等写入落盘后把记录补出来。
    setTimeout(() => {
      this.refreshData();
    }, 350);
  }
  async refreshData(): Promise<void> {
```

```
  this.sources = await Store.webdavSources();
  this.wdHist = await Store.wdHistory();
}
/** 手机扫码添加成功后：刷新源列表并给一条提示（面板里已经落盘） */
async afterQrAdd(src: WebdavSource): Promise<void> {
  await this.refreshData();
  Tip.show('已添加网盘「' + src.name + '」');
}
// ---------- 焦点 ----------
hi(key: string): boolean {
  return this.hoverKey === key;
}
hover(key: string, on: boolean): void {
  if (on) {
    this.hoverKey = key;
  } else if (this.hoverKey === key) {
    this.hoverKey = '';
  }
}
onBackPress(): boolean {
  if (this.wdQrOpen) {
    this.wdQrOpen = false;
    return true;
  }
  if (this.epOpen) {
    this.epOpen = false;
    return true;
  }
  if (this.view !== 'sources') {
    this.backToSources();
    return true;
  }
  router.back();
  return true;
}
backToSources(): void {
  this.view = 'sources';
  this.entries = [];
  this.vods = [];
  this.msg = '';
}
// ---------- 浏览 ----------
/**
 * 进入一个源：优先读上次的识别缓存，有就直接展示；没有则自动扫描识别。
 * 目录浏览不再作为默认视图，想看文件树再点「浏览文件」。
 */
async enterSource(src: WebdavSource): Promise<void> {
  this.cur = src;
  this.curPath = Webdav.mountHref(src);
  this.vods = [];
```

```
    this.msg = '';
    const cache: WebdavVod[] = await Store.webdavScanCache(src.id);
    if (cache.length > 0) {
      this.vods = cache;
      this.view = 'scan';
      this.msg = '上次识别到 ' + cache.length + ' 部影视，点「重新扫描」可刷新';
      this.msgColor = '#B7F0BC';
      return;
    }
    // 无缓存 → 切到识别视图再自动扫描（扫描中能看到进度）
    this.view = 'scan';
    await this.scanVods();
  }
  async loadDir(): Promise<void> {
    this.loading = true;
    this.entries = [];
    try {
      this.entries = await Webdav.list(this.cur, this.curPath);
    } catch (e) {
      const err = e as Error;
      this.msg = err.message;
      this.msgColor = T.WARN;
    }
    this.loading = false;
  }
  /** 从识别结果视图切回目录浏览 */
  async openBrowse(): Promise<void> {
    this.view = 'browse';
    this.msg = '';
    await this.loadDir();
  }
  async enterDir(entry: WebdavEntry): Promise<void> {
    this.curPath = Webdav.stripSlash(entry.href);
    await this.loadDir();
  }
  async goUp(): Promise<void> {
    const p: string = Webdav.stripSlash(this.curPath);
    const i: number = p.lastIndexOf('/');
    this.curPath = i <= 0 ? '/' : p.substring(0, i);
    await this.loadDir();
  }
  pathLabel(): string {
    let p: string = this.curPath;
    try {
      p = decodeURIComponent(p);
    } catch (e) {
      // 保持原文
    }
    return p;
  }
```

```
  // ---------- 播放 ----------
  playFile(entry: WebdavEntry): void {
    const v = new WebdavVod();
    v.name = Webdav.stripExt(entry.name);
    v.kind = 'movie';
    v.dir = Webdav.stripSlash(this.curPath);
    const ep = new WebdavEp();
    ep.name = entry.name;
    ep.url = Webdav.absUrl(this.cur, entry.href);
    ep.href = entry.href;
    v.eps.push(ep);
    this.playEp(v, 0);
  }
  openEp(vod: WebdavVod): void {
    if (vod.eps.length === 1) {
      this.playEp(vod, 0);
      return;
    }
    this.selVod = vod;
    this.epOpen = true;
  }
  epNameOf(vod: WebdavVod, ep: WebdavEp, i: number): string {
    if (vod.kind === 'series') {
      if (ep.season > 0 && ep.episode > 0) {
        return 'S' + WebdavPage.pad2(ep.season) + 'E' + WebdavPage.pad2(ep.episode
);
      }
      if (ep.episode > 0) {
        return '第 ' + ep.episode + ' 集';
      }
    }
    return Webdav.stripExt(ep.name);
  }
  playEp(vod: WebdavVod, idx: number): void {
    if (idx < 0 || idx >= vod.eps.length) {
      return;
    }
    // 交给播放列表，剧集可切上/下一集
    const eps: Episode[] = [];
    for (let i = 0; i < vod.eps.length; i++) {
      const e = new Episode();
      e.name = this.epNameOf(vod, vod.eps[i], i);
      e.url = vod.eps[i].url;
      eps.push(e);
    }
    Playlist.set(eps, idx, vod.name);
    this.epOpen = false;
    const p = new PlayParam();
    p.url = vod.eps[idx].url;
    p.title = vod.name + (vod.eps.length > 1 ? (' ' + eps[idx].name) : '');
```

```
  p.siteKey = 'webdav';
  p.vodId = 'webdav:' + this.cur.id + ':' + vod.name;
  p.from = 'direct';
  p.epIndex = idx;
  p.header = Webdav.authHeader(this.cur);
  // 网盘续播落盘所需字段
  p.wdSrcId = this.cur.id;
  p.wdSrcName = this.cur.name;
  p.wdVod = vod.name;
  p.wdDir = vod.dir;
  p.wdSeries = vod.eps.length > 1;
  router.pushUrl({ url: 'pages/Player', params: p });
}
// ---------- 续播 ----------
/** 从续播记录继续观看：seek 到上次进度，能重建选集时仍可切上/下一集 */
async continueWatch(h: WdHistory): Promise<void> {
  const srcs: WebdavSource[] = await Store.webdavSources();
  let src: WebdavSource | null = null;
  for (let i = 0; i < srcs.length; i++) {
    if (srcs[i].id === h.srcId) {
      src = srcs[i];
      break;
    }
  }
  if (src === null) {
    // 源都删了，这条续播记录必然失效，直接清掉
    await Store.removeWdHistory(h);
    this.wdHist = await Store.wdHistory();
    return;
  }
  // 尝试从识别缓存重建整部剧的选集，续播时也能切上/下一集
  let eps: Episode[] = [];
  let startIdx: number = 0;
  let playUrl: string = h.url;
  let title: string = h.vodName;
  const cache: WebdavVod[] = await Store.webdavScanCache(src.id);
  let vod: WebdavVod | null = null;
  for (let i = 0; i < cache.length; i++) {
    if (cache[i].name === h.vodName) {
      vod = cache[i];
      break;
    }
  }
  if (vod !== null && vod.eps.length > 0) {
    for (let i = 0; i < vod.eps.length; i++) {
      const e = new Episode();
      e.name = this.epNameOf(vod, vod.eps[i], i);
      e.url = vod.eps[i].url;
      eps.push(e);
    }
```

```
      startIdx = (h.epIndex >= 0 && h.epIndex < vod.eps.length) ? h.epIndex : 0;
      playUrl = vod.eps[startIdx].url;
      title = vod.name + (vod.eps.length > 1 ? (' ' + eps[startIdx].name) : '');
      Playlist.set(eps, startIdx, vod.name);
    } else if (h.epName.length > 0) {
      title = h.vodName + ' ' + h.epName;
    }
    const p = new PlayParam();
    p.url = playUrl;
    p.title = title;
    p.siteKey = 'webdav';
    p.vodId = 'webdav:' + src.id + ':' + h.vodName;
    p.from = 'direct';
    p.epIndex = startIdx;
    p.header = Webdav.authHeader(src);
    p.seek = h.pos;
    p.wdSrcId = src.id;
    p.wdSrcName = src.name;
    p.wdVod = h.vodName;
    p.wdDir = h.dir;
    // 是否剧集：能重建选集按集数判断；否则按历史记录粗略判断（epIndex>0 或曾有集
名）
    if (vod !== null && vod.eps.length > 0) {
      p.wdSeries = vod.eps.length > 1;
    } else {
      p.wdSeries = h.epIndex > 0 || h.epName.length > 0;
    }
    router.pushUrl({ url: 'pages/Player', params: p });
  }
  async removeWd(h: WdHistory): Promise<void> {
    await Store.removeWdHistory(h);
    this.wdHist = await Store.wdHistory();
  }
  static pad2(n: number): string {
    return n < 10 ? '0' + n : String(n);
  }
  // ---------- 识别影视 ----------
  async scanVods(): Promise<void> {
    this.loading = true;
    this.msg = '正在扫描目录识别影视资源…';
    this.msgColor = T.SUB;
    try {
      const result: WebdavVod[] = await Webdav.scan(this.cur, 6,
        (dirs: number, files: number): void => {
          this.msg = '正在扫描… 已扫 ' + dirs + ' 个目录，发现 ' + files + ' 个视
频';
        });
      this.vods = result;
      // 识别结果落盘，下次进入直接读缓存
      await Store.saveWebdavScan(this.cur.id, result);
```

```
      this.view = 'scan';
      this.msg = '识别到 ' + result.length + ' 部影视';
      this.msgColor = '#B7F0BC';
    } catch (e) {
      const err = e as Error;
      this.msg = err.message;
      this.msgColor = T.WARN;
      // 扫描失败保留在 browse 视图（如果本来就在 browse），否则停在 sources
      if (this.view !== 'browse') {
        this.view = 'browse';
        await this.loadDir();
      }
    }
    this.loading = false;
  }
  // ---------- UI ----------
  @Builder
  Btn(label: string, key: string, primary: boolean, act: () => void) {
    Text(label)
      .fontSize(15)
      .height(44)
      .padding({ left: 22, right: 22 })
      .borderRadius(T.RADIUS_PILL)
      .backgroundColor(primary ? T.ACCENT : (this.hi(key) ? T.CARD_HI : T.CARD))
      .fontColor(primary ? T.BG : (this.hi(key) ? T.SELECTED : T.TXT))
      .border({ width: T.BORDER, color: this.hi(key) ? (primary ? T.SELECTED : T.T
XT) : '#00000000' })
      .textAlign(TextAlign.Center)
      .animation({ duration: T.ANIM, curve: Curve.EaseOut })
      .focusable(true)
      .onFocus(() => this.hover(key, true))
      .onBlur(() => this.hover(key, false))
      .onHover((hv: boolean) => this.hover(key, hv))
      .onClick(() => {
        Feedback.tap();
        act();
      })
  }
  /** 通用行底板：标题 + 说明 + 右侧操作文字，焦点高亮 */
  @Builder
  RowItem(key: string, title: string, sub: string, right: string, act: () => void)
 {
    Row() {
      Column() {
        Text(title).fontSize(15).fontColor(T.TXT).maxLines(1).textOverflow({ overf
low: TextOverflow.Ellipsis })
        if (sub.length > 0) {
          Text(sub).fontSize(12).fontColor(T.DIM).margin({ top: 3 }).maxLines(1)
            .textOverflow({ overflow: TextOverflow.Ellipsis })
        }
```

```
    }
    .alignItems(HorizontalAlign.Start)
    .layoutWeight(1)
    if (right.length > 0) {
      Text(right).fontSize(13).fontColor(T.SELECTED).margin({ left: 10 })
    }
  }
  .width('100%')
  .height(56)
  .padding({ left: 14, right: 14 })
  .borderRadius(T.RADIUS_SM)
  .backgroundColor(this.hi(key) ? T.CARD_HI : T.CARD)
  .border({ width: T.BORDER, color: this.hi(key) ? T.TXT : '#00000000' })
  .margin({ bottom: 8 })
  .animation({ duration: T.ANIM, curve: Curve.EaseOut })
  .focusable(true)
  .onFocus(() => this.hover(key, true))
  .onBlur(() => this.hover(key, false))
  .onHover((hv: boolean) => this.hover(key, hv))
  .onClick(() => {
    Feedback.tap();
    act();
  })
}
@Builder
TopBar(title: string) {
  Row() {
    Text('‹')
      .fontSize(34)
      .fontColor(this.hi('back') ? T.TXT : T.SUB)
      .width(40)
      .textAlign(TextAlign.Center)
      .focusable(true)
      .defaultFocus(true)
      .onFocus(() => this.hover('back', true))
      .onBlur(() => this.hover('back', false))
      .onHover((hv: boolean) => this.hover('back', hv))
      .onClick(() => {
        Feedback.tap();
        if (this.view === 'sources') {
          router.back();
        } else {
          this.backToSources();
        }
      })
    Text(title).fontSize(24).fontColor(T.TXT).margin({ left: 8 })
    if (this.view === 'browse') {
      Text(this.cur.name + '  ·  ' + this.pathLabel())
        .fontSize(14).fontColor(T.DIM).margin({ left: 16 }).maxLines(1)
        .textOverflow({ overflow: TextOverflow.Ellipsis }).layoutWeight(1)
```

```
      } else if (this.view === 'scan') {
        Text(this.cur.name)
          .fontSize(14).fontColor(T.DIM).margin({ left: 16 }).maxLines(1)
          .textOverflow({ overflow: TextOverflow.Ellipsis }).layoutWeight(1)
      }
    }
    .width('100%')
    .padding({ left: 24, right: 24, top: 24, bottom: 8 })
  }
  build() {
    Stack({ alignContent: Alignment.Center }) {
      Column() {
        this.TopBar(this.view === 'sources' ? 'WebDAV 网盘' : (this.view === 'brow
se' ? '浏览文件' : '识别影视'))
        Scroll() {
          Column() {
            if (this.view === 'sources') {
              this.sourcesView()
            } else if (this.view === 'browse') {
              this.browseView()
            } else {
              this.scanView()
            }
          }
          .width('100%')
          .padding({ left: 24, right: 24, bottom: 30 })
          .alignItems(HorizontalAlign.Start)
        }
        .layoutWeight(1)
        .width('100%')
        .scrollBar(BarState.Off)
      }
      .width('100%')
      .height('100%')
      .backgroundColor(T.pageBg(this.themeMode))
      if (this.epOpen) {
        this.epPanel()
      }
      if (this.wdQrOpen) {
        WdQrPanel({
          onClose: () => {
            this.wdQrOpen = false;
          },
          onSaved: (src: WebdavSource) => {
            this.wdQrOpen = false;
            this.afterQrAdd(src);
          }
        })
      }
      if (this.tipMsg.length > 0) {
```

```
        TipLayer({ msg: this.tipMsg })
      }
    }
    .width('100%')
    .height('100%')
    .backgroundColor(T.pageBg(this.themeMode))
    // id 带主题名：让 build 真正依赖 themeMode（否则换色板不重绘）
    .id('webdav_' + this.themeMode)
  }
  @Builder
  sourcesView() {
    Column() {
      // 继续观看：网盘播放记录，点击从上次进度续播
      if (this.wdHist.length > 0) {
        Row() {
          Text('继续观看').fontSize(17).fontColor(T.TXT)
          Blank()
          Text('清空').fontSize(13).fontColor(T.DIM)
            .padding({ left: 12, right: 12, top: 6, bottom: 6 })
            .borderRadius(T.RADIUS_PILL)
            .backgroundColor(this.hi('clearWd') ? T.CARD_HI : T.CARD)
            .border({ width: T.BORDER, color: this.hi('clearWd') ? T.TXT : '#00000
000' })
            .focusable(true)
            .onFocus(() => this.hover('clearWd', true))
            .onBlur(() => this.hover('clearWd', false))
            .onHover((hv: boolean) => this.hover('clearWd', hv))
            .onClick(() => {
              Feedback.tap();
              this.clearWd();
            })
        }
        .width('100%')
        .margin({ top: 6, bottom: 8 })
        ForEach(this.wdHist, (h: WdHistory) => {
          this.wdHistItem(h)
        }, (h: WdHistory) => h.srcId + '_' + h.vodName)
        Text('挂载的网盘在这里浏览目录、识别并播放影视资源；新增 / 编辑 / 删除网盘
请到「设置 → 网盘」。')
          .fontSize(14).fontColor(T.SUB).width('100%').lineHeight(22).margin({ top
: 18 })
      } else {
        Text('挂载的网盘在这里浏览目录、识别并播放影视资源；新增 / 编辑 / 删除网盘
请到「设置 → 网盘」。')
          .fontSize(14).fontColor(T.SUB).width('100%').lineHeight(22)
      }
      Row() {
        // 电视端首选：扫码在手机上填，遥控器只负责点这一下
        if (!this.isPhone) {
          this.Btn('扫码添加网盘', 'qr_add', true, () => {
```

```
            this.wdQrOpen = true;
          })
        }
        this.Btn('去设置页添加', 'go_setting', this.isPhone, () => {
          router.pushUrl({ url: 'pages/SettingPage' });
        })
        Blank()
      }
      .width('100%')
      .margin({ top: 16, bottom: 6 })
      if (this.sources.length === 0) {
        Text(this.isPhone ? '还没有挂载任何网盘。点「去设置页添加」，填服务器地址
、账号密码即可。'
          : '还没有挂载任何网盘。点「扫码添加网盘」，用手机扫码填服务器地址和账号
密码，电视这台不用敲字。')
          .fontSize(14).fontColor(T.DIM).width('100%').margin({ top: 20 }).lineHei
ght(22)
      } else {
        ForEach(this.sources, (s: WebdavSource) => {
          this.RowItem('s_' + s.id, s.name, s.url, '进入 ›', () => {
            this.enterSource(s);
          })
        }, (s: WebdavSource) => s.id)
      }
    }
  }
  /** 单条续播记录 */
  @Builder
  wdHistItem(h: WdHistory) {
    Row() {
      Column() {
        Text(h.vodName).fontSize(15).fontColor(T.TXT).maxLines(1)
          .textOverflow({ overflow: TextOverflow.Ellipsis })
        Row() {
          Text(h.epName.length > 0 ? ('看到 ' + h.epName) : '电影')
            .fontSize(12).fontColor(T.DIM)
          if (h.pos > 0) {
            Text(' · 已看 ' + WebdavPage.fmtPos(h.pos) + (h.dur > 0 ? (' / ' + Web
davPage.fmtPos(h.dur)) : ''))
              .fontSize(12).fontColor(T.SELECTED)
          }
        }
        .margin({ top: 3 })
      }
      .alignItems(HorizontalAlign.Start)
      .layoutWeight(1)
      .onClick(() => {
        Feedback.tap();
        this.continueWatch(h);
      })
```

```
      Text('继续播放 ›')
        .fontSize(13).fontColor(T.SELECTED).margin({ left: 10 })
        .onClick(() => {
          Feedback.tap();
          this.continueWatch(h);
        })
      Text('×')
        .fontSize(15).fontColor(T.DIM)
        .width(36).height(36).textAlign(TextAlign.Center).borderRadius(18)
        .margin({ left: 8 })
        .onClick(() => {
          Feedback.tap();
          this.removeWd(h);
        })
    }
    .width('100%')
    .height(56)
    .padding({ left: 14, right: 14 })
    .borderRadius(T.RADIUS_SM)
    .backgroundColor(this.hi('w_' + h.srcId + '_' + h.vodName) ? T.CARD_HI : T.CAR
D)
    .border({ width: T.BORDER, color: this.hi('w_' + h.srcId + '_' + h.vodName) ?
T.TXT : '#00000000' })
    .margin({ bottom: 8 })
    .animation({ duration: T.ANIM, curve: Curve.EaseOut })
    .focusable(true)
    .onFocus(() => this.hover('w_' + h.srcId + '_' + h.vodName, true))
    .onBlur(() => this.hover('w_' + h.srcId + '_' + h.vodName, false))
    .onHover((hv: boolean) => this.hover('w_' + h.srcId + '_' + h.vodName, hv))
  }
  async clearWd(): Promise<void> {
    await Store.clearWdHistory();
    this.wdHist = [];
  }
  @Builder
  browseView() {
    Column() {
      Row() {
        this.Btn('识别影视', 'scan', true, () => {
          this.scanVods();
        })
        this.Btn('上级目录', 'up', false, () => {
          this.goUp();
        })
        this.Btn('刷新', 'refresh', false, () => {
          this.loadDir();
        })
        Blank()
      }
      .width('100%')
```

```
      .margin({ top: 8, bottom: 12 })
      if (this.loading) {
        Row() {
          LoadingProgress().width(24).height(24).color(T.SUB)
          Text('加载中…').fontSize(14).fontColor(T.DIM).margin({ left: 10 })
        }
        .width('100%').margin({ top: 20 })
      } else if (this.entries.length === 0) {
        Text(this.msg.length > 0 ? this.msg : '这个目录是空的。')
          .fontSize(14).fontColor(this.msgColor).width('100%').margin({ top: 16 })
.lineHeight(22)
      } else {
        ForEach(this.entries, (e: WebdavEntry, i: number) => {
          this.RowItem(
            'f_' + i,
            (e.isDir ? '📁 ' : '🎬 ') + e.name,
            e.isDir ? '目录' : (WebdavPage.sizeLabel(e.size) + (e.mtime.length > 0
 ? ' · ' + e.mtime : '')),
            e.isDir ? '进入' : '播放',
            () => {
              if (e.isDir) {
                this.enterDir(e);
              } else if (Webdav.isVideo(e.name)) {
                this.playFile(e);
              }
            })
        }, (e: WebdavEntry, i: number) => e.href + i)
      }
    }
  }
  @Builder
  scanView() {
    Column() {
      Row() {
        this.Btn('浏览文件', 'browse', false, () => {
          this.openBrowse();
        })
        this.Btn('重新扫描', 'rescan', true, () => {
          this.scanVods();
        })
        Blank()
      }
      .width('100%')
      .margin({ top: 8, bottom: 12 })
      if (this.loading) {
        Column() {
          Row() {
            LoadingProgress().width(24).height(24).color(T.SUB)
            Text('扫描中…').fontSize(14).fontColor(T.DIM).margin({ left: 10 })
          }
```

```
          if (this.msg.length > 0) {
            Text(this.msg).fontSize(13).fontColor(T.DIM).width('100%').margin({ to
p: 10 }).lineHeight(20)
          }
        }
        .width('100%').margin({ top: 20 })
      } else if (this.vods.length === 0) {
        Text(this.msg.length > 0 ? this.msg : '没有识别到视频资源。')
          .fontSize(14).fontColor(this.msgColor).width('100%').margin({ top: 16 })
.lineHeight(22)
      } else {
        ForEach(this.vods, (v: WebdavVod, i: number) => {
          this.RowItem('v_' + i, v.name, this.vodMeta(v), '播放', () => {
            this.openEp(v);
          })
        }, (v: WebdavVod, i: number) => v.name + i)
      }
    }
  }
  @Builder
  epPanel() {
    Stack() {
      Column()
        .width('100%').height('100%')
        .backgroundColor(T.BLACK_70)
        .onClick(() => {
          Feedback.tap();
          this.epOpen = false;
        })
      Column() {
        Row() {
          Text(this.selVod.name).fontSize(19).fontColor(T.TXT).maxLines(1)
            .textOverflow({ overflow: TextOverflow.Ellipsis }).layoutWeight(1)
          Blank()
          Text('×').fontSize(19).fontColor(this.hi('ex') ? T.TXT : T.SUB)
            .width(42).height(42).textAlign(TextAlign.Center).borderRadius(T.RADIU
S_SM)
            .backgroundColor(this.hi('ex') ? T.CARD_HI : '#00000000')
            .focusable(true).defaultFocus(true)
            .onFocus(() => this.hover('ex', true))
            .onBlur(() => this.hover('ex', false))
            .onHover((hv: boolean) => this.hover('ex', hv))
            .onClick(() => {
              Feedback.tap();
              this.epOpen = false;
            })
        }
        .width('100%')
        Scroll() {
          Column() {
```

```
            ForEach(this.selVod.eps, (ep: WebdavEp, i: number) => {
              this.RowItem('ep_' + i, this.epNameOf(this.selVod, ep, i), '', '播放
', () => {
                this.playEp(this.selVod, i);
              })
            }, (ep: WebdavEp, i: number) => ep.href + i)
          }
          .width('100%')
        }
        .width('100%')
        .height(this.isPhone ? 320 : 360)
        .margin({ top: 14 })
        .scrollBar(BarState.Off)
      }
      .width(Device.panelW(520))
      .padding(24)
      .backgroundColor(T.SURFACE)
      .borderRadius(28)
      .alignItems(HorizontalAlign.Start)
      .focusScopeId('ep_panel', true)
    }
    .width('100%')
    .height('100%')
  }
  vodMeta(v: WebdavVod): string {
    return (v.kind === 'series' ? '剧集 · ' : '电影 · ') +
      (v.year.length > 0 ? v.year + ' · ' : '') + v.eps.length + ' 集';
  }
  static sizeLabel(size: number): string {
    if (size <= 0) {
      return '';
    }
    if (size >= 1024 * 1024 * 1024) {
      return (size / 1024 / 1024 / 1024).toFixed(2) + ' GB';
    }
    if (size >= 1024 * 1024) {
      return (size / 1024 / 1024).toFixed(1) + ' MB';
    }
    if (size >= 1024) {
      return (size / 1024).toFixed(0) + ' KB';
    }
    return size + ' B';
  }
  static fmtPos(ms: number): string {
    if (ms <= 0) {
      return '0:00';
    }
    const t: number = Math.floor(ms / 1000);
    const h: number = Math.floor(t / 3600);
    const m: number = Math.floor((t % 3600) / 60);
```

```
    const s: number = t % 60;
    const mm: string = (m < 10 ? '0' : '') + m;
    const ss: string = (s < 10 ? '0' : '') + s;
    return h > 0 ? (h + ':' + mm + ':' + ss) : (mm + ':' + ss);
  }
}
// ================================================================================
// 源程序结束
// 软件名称：夜帧影视软件     版本号：V1.0
// 开发完成日期：2026 年 11 月 04 日
// ================================================================================
//
//
//
```